STTN327 | Project report

Predicting sulphur dioxide in a US city that is not in the data

Data set: SO2 air pollution in 41 US cities, 1969–1971

North-West University | October 2026

TABLE OF CONTENTS

1 Introduction 2

1.1 Scenario 2

1.2 Research questions 2

1.3 Type of design 2

1.4 Data 2

2 Exploring the relationships

3 How we test a model 6

4 The four models 8

4.1 Why four models 8

4.2 Multiple linear regression 8

4.3 Ridge and Lasso regression 10

4.4 Random forest 12

4.5 Mixed-effects model 13

5 Comparing the models 16

6 Using the model on a new city 17

7 Conclusion 18

7.1 Answers to the research questions 18

7.2 Limitations 18

Appendix: Code 20

1 INTRODUCTION

1.1 Scenario

During the 1970s there was serious concern about the amount of sulphur dioxide (SO2) in the atmosphere of the United States, because it was thought to contribute to so-called “acid rain”. A study collected monthly air-pollution data from 41 large US cities from January 1969 to December 1971 (36 months per city). For every city-month it recorded the mean SO2 concentration (μg/m³), the mean temperature (°F), the mean wind speed (mph), the total precipitation (inches) and the number of days with precipitation. Two variables describing the city itself were also recorded: the number of manufacturing enterprises employing 20 or more workers, and the population in 1970 (in thousands).

The brief asks us to explore the relationships between these variables and to attempt to build a predictive model for the level of SO2 in the atmosphere.

1.2 Research questions

The main question we chose to answer is:

> ***Can quarterly SO2 levels be predicted from weather, season, city size and industry, including for a city that is not in our data?***

We answer it in three parts:

- **RQ1:** What drives SO2 in the 41 cities?
- **RQ2:** How well can SO2 be predicted for a new city, with no readings from it, and with one or two quarters of its readings?
- **RQ3:** Which of four modelling approaches (multiple linear regression, Ridge/Lasso regression, random forest, mixed-effects model) predicts best?

Testable aim. A model succeeds if its prediction error for a city it has never seen is clearly below the error of the benchmark: guessing the average SO2 of the other 40 cities (RMSE 23.98 μg/m³). Where a hypothesis test is used we work at the 5% level of significance.

This is not time-series forecasting. We predict SO2 from a city’s conditions, not future years from past years.

1.3 Type of design

The study is observational: nobody assigned cities to levels of temperature or industry, so every relationship we find is an association and not proof of cause. The unit of observation is the city. Each city is measured repeatedly (12 quarters), so the 492 rows are not 492 independent observations: the 12 rows of one city are strongly related. Two consequences shape everything that follows: standard errors are clustered by city, and every model is tested on whole cities that it was not fitted to.

1.4 Data

All models use one quarterly file, quarterly_SO2_data.csv, with 492 rows (41 cities × 12 quarters) and no missing values. It was built from the original monthly Excel file as follows:

- Missing monthly values (about 6.6% of city-months had at least one) were filled by linear interpolation along that city's own series; at the start or end of a series the city's own mean was used.
- Months were grouped into quarters (Q1 = January to March, and so on).
- SO2, temperature, wind speed and precipitation were averaged within each quarter; precipitation days were summed.
- Manufacturing enterprises and population were joined in from the census sheet.

We chose quarters because months are noisy and have many gaps, while years would leave only three rows per city. Quarter was coded as three indicator variables (Q2, Q3, Q4) with Q1 as the reference.

Table 1. Summary of the variables (492 city-quarter rows)

Variable	Mean	SD	Minimum	Maximum
SO2 (µg/m³)	30.1	23.4	3.5	114.7
Temperature (°F)	55.8	11.6	26.5	91.8
Wind speed (mph)	12.9	15.2	5.6	92.6
Precipitation (inches)	36.8	11.7	5.3	61.4
Precipitation days	28.5	7.5	5	47
Manufacturing enterprises	463.1	557.1	35	3344
Population (thousands)	608.6	572.6	71	3369

Data quality: impossible wind speeds

Memphis reports a wind speed of about 92 mph and Charleston about 65 mph in every quarter, while a typical city is between 5 and 15 mph. These are recording errors. We kept both cities, but checked what they do. Removing them changes the wind-speed coefficient in the regression from −0.085 to −2.6 (clustered p = 0.10), so that coefficient is not reliable. The new-city prediction error of the regression, Ridge, Lasso and mixed-effects models changes by 0.15 or less, so predictions do not depend on them. The random forest was fitted without wind speed for this reason (Section 4.4).

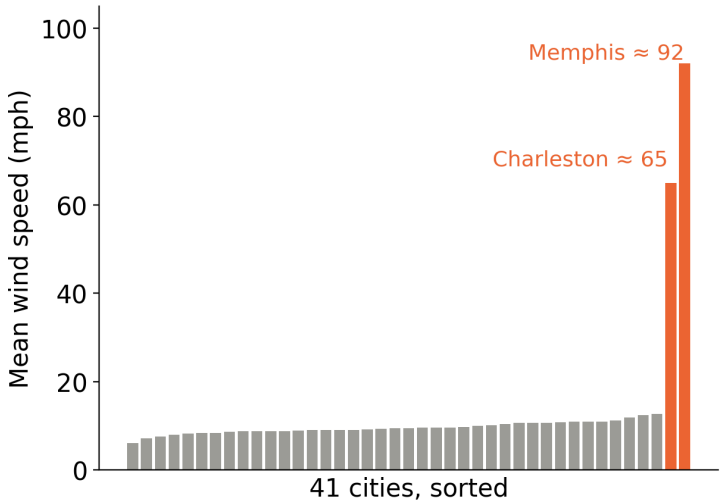


Figure 1. Mean wind speed of the 41 cities, sorted. Memphis and Charleston stand out.

2 EXPLORING THE RELATIONSHIPS

Before modelling we looked at how the variables relate to SO2 and to each other (Figure 2). SO2 is most strongly related to the number of manufacturing enterprises (r = 0.64) and to population (r = 0.49). Temperature is the clearest weather variable: colder quarters go with more SO2 (r = −0.37). Precipitation days (r = 0.31) is weakly positive, while wind speed (r = −0.10) and the amount of precipitation (r = 0.05) show little relationship on their own.

Manufacturing and population are almost the same information (r = 0.96): big cities have many factories. This collinearity means their coefficients cannot be read one at a time, a point we return to in the models.

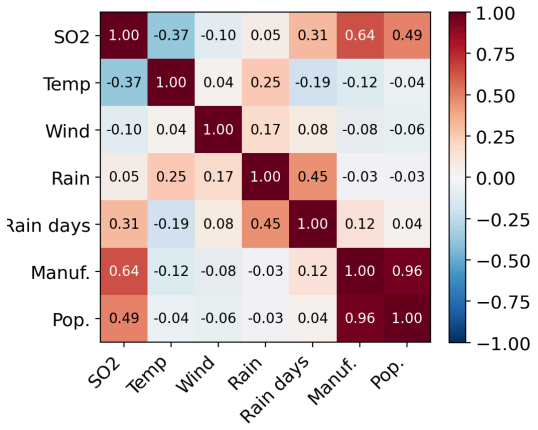


Figure 2. Correlation matrix of SO2 and the six predictors (492 city-quarters).

The scatter plot of city mean SO2 against manufacturing (Figure 3) shows that more industrial cities have higher SO2, and that a few cities (Providence, Philadelphia, Chicago) are far from the rest. The boxplots (Figure 4) show that SO2 changes only slightly from quarter to quarter compared with the differences between cities. This already suggests that most of the variation is between cities, not within them, and that predicting a new city is mostly about predicting its overall level.

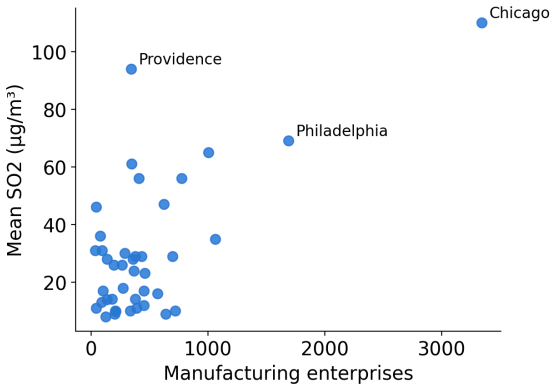


Figure 3. Mean SO2 of each city against its number of manufacturing enterprises.

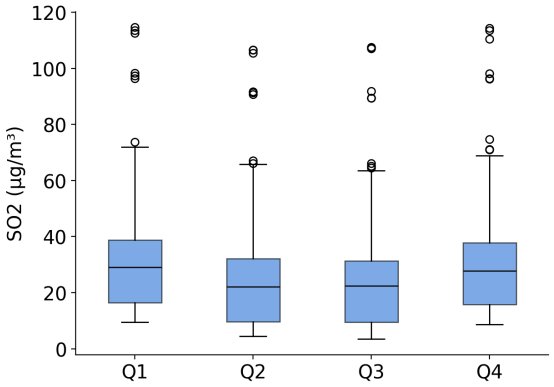


Figure 4. SO2 by quarter (all cities).

3 HOW WE TEST A MODEL

Our question is about a city that is not in the data, so a model must be judged on cities it has not seen. We use leave-one-city-out cross-validation: for each of the 41 cities we fit the model on the other 40 and predict all 12 quarters of the city that was left out. Every one of the 492 predictions therefore comes from a model that never saw that city. Any tuning (for example the Ridge penalty or the random-forest settings) is done inside the 40 training cities only.

We report three situations, identified by k, the number of the new city's quarters whose real SO2 we are allowed to see:

- **k = 0:** no SO2 readings from the city. This is the hardest case and the one in our question.
- **k = 1:** the first quarter is revealed; we add the miss on that quarter to the predictions for the remaining 11 quarters.
- **k = 2:** the first two quarters are revealed; we add the average miss on those two to the remaining 10 quarters.

Only the quarters that were not revealed are scored. We report two measures over all held-out predictions pooled together:

- **RMSE** (root mean squared error): the typical size of a prediction error, in µg/m³. Lower is better.
- **Pooled R²** = 1 − (sum of squared errors of the model) / (sum of squared errors of the benchmark), where the benchmark guesses the mean SO2 of the other 40 cities. It is the share of the variation explained, relative to guessing the average.

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum(y-\hat{y})^2}$$

$$R^2 = 1 - \frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2}$$

Here y is the observed SO2, ŷ is the model's prediction, ȳ is the benchmark (the mean SO2 of the other 40 cities), n is the number of held-out quarters scored, and each sum runs over those quarters.

Because the 41 cities, not the 492 rows, are the independent units, confidence intervals come from resampling whole cities 10,000 times (a city-level bootstrap). The benchmark has an RMSE of 23.98.

Why not a random split of the rows?

The 12 quarters of one city are very similar. If a model is scored on random rows it has already seen the other quarters of the same city and looks almost perfect. Figure 5 shows the same random forest scored four ways: scored on random rows (out-of-bag and 5-fold) it reaches R² of about 0.97, but scored on a city it has not seen its R² is 0.37 after tuning and below zero before tuning. Only the leave-one-city-out score answers our research question, so it is the one used for every model.

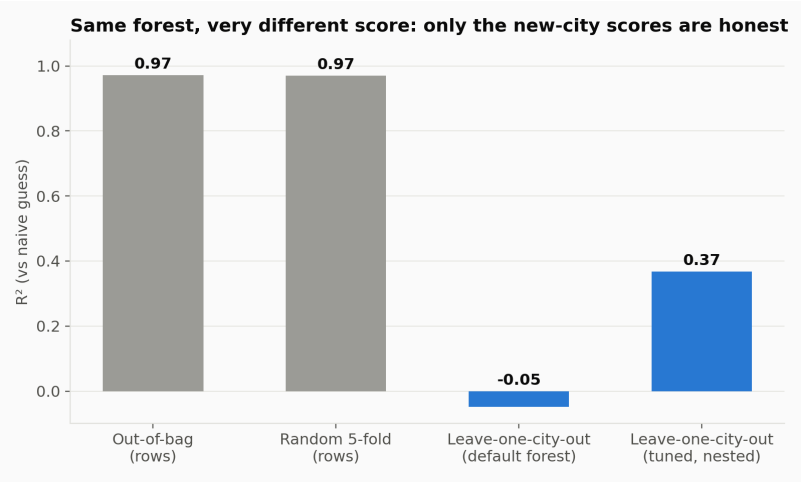


Figure 5. The same forest scored four ways. Only the new-city scores are honest.

4 THE FOUR MODELS

4.1 Why four models

Each of the four models answers a different weakness in the data, and all four are scored in exactly the same way, so the comparison is fair. If four different approaches reach the same answer, the answer is not an accident of one model choice; where they disagree, we learn why.

Model	What it does	Question it answers
Multiple linear regression	Fits the best straight-line relationship between SO2 and all predictors at once.	What does the plain linear story say?
Ridge and Lasso	Regression with a penalty that shrinks coefficients; Lasso can set some to exactly zero.	Do overfitting or collinearity hurt?
Random forest	Hundreds of decision trees, each on a random sample, vote; averaging cancels out their errors.	Are we missing curved or interacting effects?
Mixed-effects model	Regression plus a personal baseline (random intercept) for every city.	Does the city structure matter?

4.2 Multiple linear regression

Model. With Q1 as the reference quarter:

$$y = \beta_0 + \beta_1 T + \beta_2 W + \beta_3 R + \beta_4 D$$

$$+ \beta_5 M + \beta_6 P + \beta_7 Q_2 + \beta_8 Q_3 + \beta_9 Q_4 + \varepsilon$$

Notation used in the equations of this report

Symbol	Meaning
y	Quarterly SO2 of a city (µg/m³); ŷ is its prediction
T	Temperature (°F)
W	Wind speed (mph)
R	Precipitation (inches)
D	Precipitation days
M	Manufacturing enterprises (20+ workers)
P	Population in 1970 (thousands)
Q2, Q3, Q4	Quarter indicators: 1 in that quarter, 0 otherwise (Q1 is the baseline)
β0, …, β9	Coefficients estimated from the data; ε is the random error

Predictors were chosen to match the question: weather (temperature, wind speed, precipitation, precipitation days), season (quarter indicators) and city size and industry (population, manufacturing enterprises). Fitted on all 41 cities the model has $R^2 = 0.641$ (adjusted 0.634), $F(9, 482) = 95.5$, $p < 0.001$. This in-sample fit is higher than the new-city R^2 below, because it includes the cities being predicted.

Testing the coefficients

For each predictor we test:

$$H_0: \beta_j = 0 \quad \text{(the predictor has no effect on SO2, other predictors held fixed)}$$
$$H_A: \beta_j \neq 0$$

Table 2 gives two p-values. The “naive” column treats the 492 rows as independent and is not valid, because rows from one city are strongly related (Durbin-Watson = 0.147, far from 2). The “clustered” column uses standard errors clustered by city and is the one we use.

Table 2. Regression coefficients and p-values

Predictor	Estimate	p (naive)	p (clustered by city)
Intercept	43.406	< 0.001	< 0.001
Temperature (per °F)	−0.581	< 0.001	0.010
Wind speed (per mph)	−0.085	0.049	0.206
Precipitation (per inch)	0.278	< 0.001	0.199
Precipitation days	0.123	0.288	0.714
Manufacturing enterprises	0.070	< 0.001	< 0.001
Population (thousands)	−0.046	< 0.001	0.002
Q2 (vs Q1)	3.09	0.207	0.401
Q3 (vs Q1)	3.45	0.172	0.382
Q4 (vs Q1)	0.58	0.750	0.234

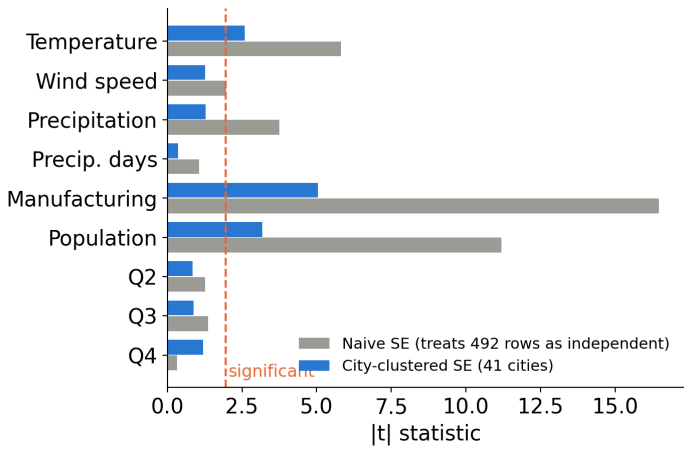


Figure 6. |t| statistics with naive and city-clustered standard errors. Orange line: 5% significance.

Interpretation. After clustering by city, three predictors are significant at 5%: temperature, manufacturing and population. Each additional °F is associated with 0.58 µg/m³ less SO2. Manufacturing (+0.070 per enterprise) and population (−0.046 per thousand people) have opposite signs, but they are 96% correlated (VIF 13.7 and 13.3), so they must be read together: a city with many factories for its size has more SO2. Wind speed, precipitation, precipitation days and the quarter indicators are not significant once the clustering is accounted for. Precipitation and wind speed look significant only because the naive test overstates certainty.

Checking the assumptions

Normality of the residuals:

$$H_0: \text{the errors come from a normal population}$$
$$H_A: \text{the errors are not normal}$$

The Shapiro-Wilk p-value is below 0.001 (skewness 0.87), so we reject H0: the residuals are right-skewed. Equal variance:

$$H_0: \text{the error variance is constant}$$
$$H_A: \text{the error variance is not constant}$$

The Breusch-Pagan p-value is below 0.001, so we reject H0. Independence fails as well (Durbin-Watson above). The most important finding, shown in Figure 7, is that about 98% of the leftover error is a city-wide offset: a city is over-predicted or under-predicted in all of its quarters. The assumptions therefore do not hold in the strict sense, which is why we use clustered standard errors for inference, and why a model with a baseline for each city (Section 4.5) is worth trying. The errors do not make the predictions invalid; they limit how far plain regression can go for a new city.

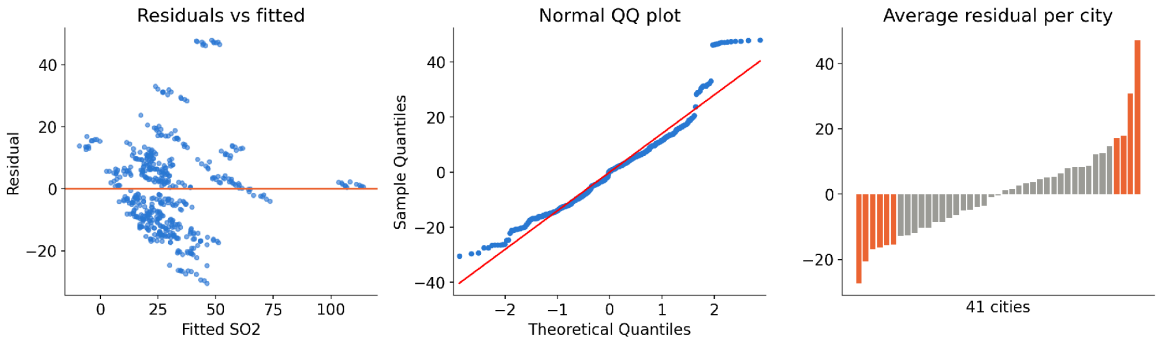


Figure 7. Residuals against fitted values, normal QQ plot, and average residual of each city.

Robustness. Chicago is a high-leverage city (R^2 is 0.641 with it and 0.493 without it). Excluding Memphis and Charleston changes the wind-speed coefficient from −0.085 to −2.6 (Section 1.4) but not the predictions.

Prediction for a new city

Table 3. Regression, leave-one-city-out

Information about the new city	Pooled RMSE	Pooled R^2
k = 0 (none)	16.03	0.553
k = 1 (first quarter)	2.75	0.987
k = 2 (two quarters)	2.09	0.992

With no data from the city the regression reduces the error from 23.98 to 16.03 and explains 55% of the variation relative to guessing the average.

4.3 Ridge and Lasso regression

Ridge and Lasso fit the same regression but add a penalty that shrinks the coefficients towards zero, to protect against overfitting. Ridge shrinks all coefficients smoothly; Lasso can shrink some

of them to exactly zero. The strength of the penalty (α) is chosen by 5-fold cross-validation that keeps whole cities together, inside the 40 training cities of every leave-one-city-out fold. Predictors are standardised first. In the 41 leave-one-city-out fits the chosen α had a median of 11.5 for Ridge and 0.20 for Lasso.

Fitted on all 41 cities (Ridge α = 11.5, Lasso α = 0.15) the coefficients, in original units, are:

Table 4. Coefficients from regression, Ridge and Lasso

Predictor	Regression	Ridge	Lasso
Temperature	−0.581	−0.653	−0.527
Wind speed	−0.085	−0.095	−0.076
Precipitation	0.278	0.249	0.229
Precipitation days	0.123	0.245	0.192
Manufacturing enterprises	0.070	0.048	0.064
Population (thousands)	−0.046	−0.025	−0.040
Q2	3.09	3.94	1.40
Q3	3.45	4.40	1.71
Q4	0.58	0.61	0.00

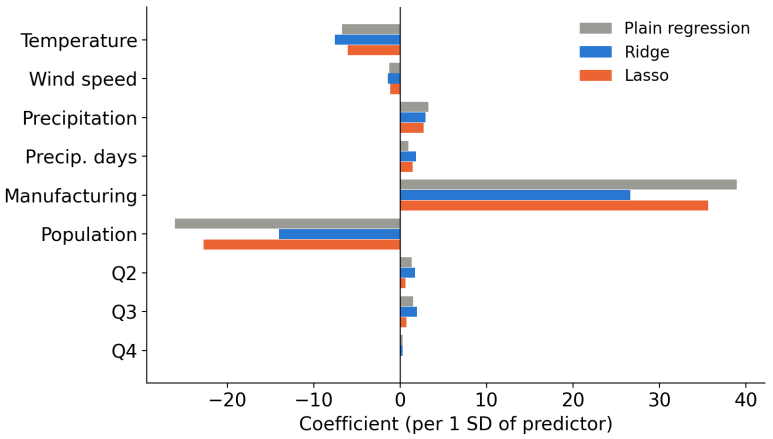


Figure 8. Coefficients with every predictor on the same scale (per 1 SD). Ridge pulls manufacturing and population together; Lasso zeroes only Q4.

Interpretation. The visible effect of Ridge is on the correlated pair: it pulls manufacturing down (from 0.070 to 0.048) and population towards zero (from −0.046 to −0.025), because it does not trust two near-duplicate predictors to carry large opposite weights. Lasso sets only the Q4 indicator to exactly zero and keeps the rest. Temperature barely changes.

Table 5. Ridge and Lasso, leave-one-city-out

Model	k = 0 RMSE	k = 0 R^2	k = 1 RMSE	k = 2 RMSE
Lasso	16.10	0.549	2.64	2.04
Ridge	16.39	0.533	3.23	2.55
Regression (for reference)	16.03	0.553	2.75	2.09

Even though the coefficients change, the predictions are no better than plain regression. With 41 cities and only nine predictors there is little overfitting to remove, so penalising changes how we explain the model but not how well it predicts for a new city.

4.4 Random forest

A random forest builds hundreds of decision trees, each on a bootstrap sample of the data and each allowed to look at only a few randomly chosen predictors at every split, and averages their predictions. A tree can only predict values that are averages of cities it has already seen, which matters for very large or very polluted cities.

Set-up

Predictors were temperature, precipitation, precipitation days, manufacturing enterprises, population and the three quarter indicators. Wind speed was left out on purpose, because Memphis and Charleston report impossible values in every quarter and a forest could use them to recognise those two cities. A check showed the default forest scores a new-city RMSE of 25.5 with wind speed and 24.6 without it.

The settings min_samples_leaf (1, 3, 5, 10, 20) and max_features (2, 3, 5) were chosen by 5-fold cross-validation split by city, repeated inside the 40 training cities of every leave-one-city-out fold (nested cross-validation), so the held-out city never influences the settings. On all 41 cities the best setting was min_samples_leaf = 10 and max_features = 3 (inner RMSE 19.3). Letting each split look at 5 predictors was clearly worse (about 21).

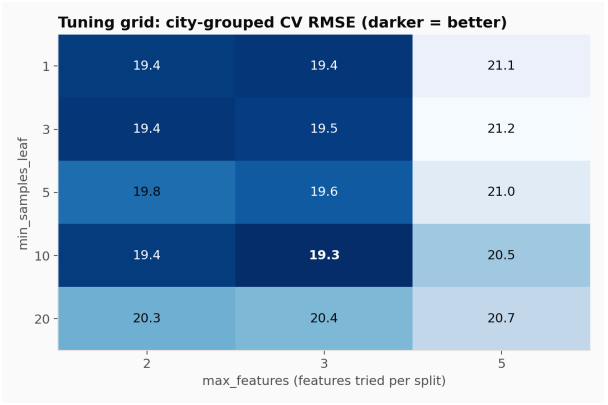


Figure 9. City-grouped cross-validation RMSE for each setting (darker is better).

What the forest relies on

Importance was measured by shuffling a predictor in held-out cities and recording how much the error rises (permutation importance), with cities kept together in the folds. Manufacturing and population were also shuffled together as one block because they are 96% correlated. City size and industry together raise the error by 2.9, precipitation (amount and days) by 2.2, temperature by 1.3, and season by about zero. This agrees with the regression on industry and size, and on season adding nothing. The forest also leans on precipitation; since precipitation differs a lot between cities but hardly changes within one, it probably stands in for where a city is, and not for a cause of SO2. That is an interpretation, not something this data can prove. The error bars for the industry block are wide, so the order of the top three should not be over-read.

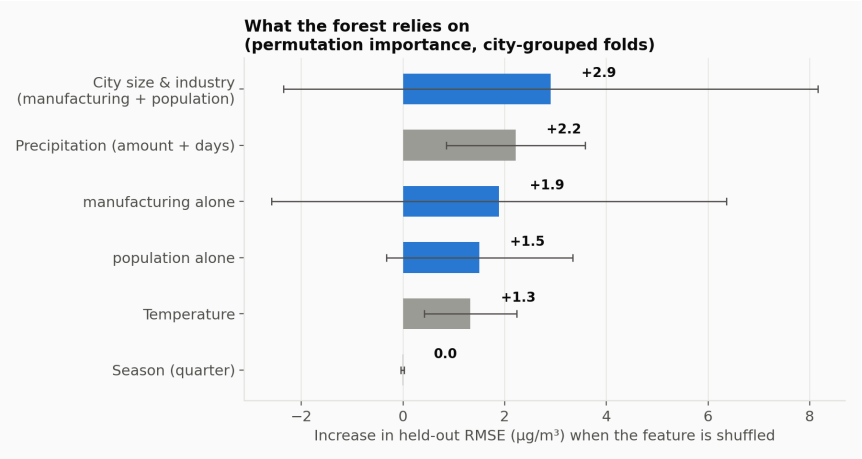


Figure 10. Increase in held-out RMSE when a predictor or block is shuffled (mean and SD over 10 city-grouped folds).

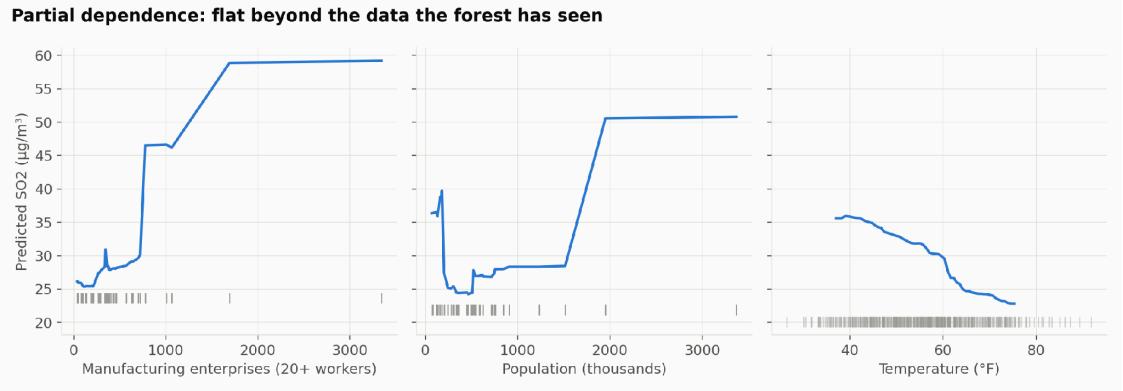


Figure 11. Partial dependence: predicted SO2 goes flat beyond the largest cities in the data.

Where it fails. The forest is too low for the heavy-SO2 cities: Providence (+61), Chicago (+51), Pittsburgh (+34), St. Louis (+31), Cleveland (+28) and Philadelphia (+24). It cannot predict above the highest values of cities it has seen, and Chicago is the only city with a population above 2 million: when Chicago is left out, nothing in the training data resembles it. Figure 11 shows the same limit.

Table 6. Random forest, leave-one-city-out

Information about the new city	RMSE	R^2	95% CI for RMSE
k = 0 (none)	19.09	0.367	13.3 to 24.7
k = 1 (first quarter)	4.05	0.971	3.4 to 4.7
k = 2 (two quarters)	3.45	0.979	3.0 to 4.0

With no data from the city the forest cuts the error from 24.0 to 19.1 and explains about 37% of the variation (95% interval for R^2: 0.13 to 0.54). It is the weakest of the four models, mainly because it cannot extrapolate to extreme cities.

4.5 Mixed-effects model

The mixed-effects model gives every city its own baseline SO2 level (a random intercept) and shares the weather, season and industry effects across all cities. For city i = 1, …, 41 and quarter j = 1, …, 12:

$$y_{ij} = \beta_0 + \beta_1 T_{ij} + \cdots + \beta_9 Q_{4,ij} + u_i + \varepsilon_{ij}$$

$$u_i \sim N(0, \tau^2), \qquad \varepsilon_{ij} \sim N(0, \sigma^2)$$

Here u_i is the baseline shift of city i (i = 1, …, 41), ε_{ij} is quarter-to-quarter noise (j = 1, …, 12), τ² is the variance between cities and σ² the variance within a city. The model was fitted by REML (restricted maximum likelihood) on all 41 cities. The fixed part of the model is the same equation we can compute for any city from its own variables alone; we call it the SO2 Index.

Table 7. Fixed effects of the mixed-effects model

Predictor	Estimate	SE	p-value
Intercept	38.076	4.112	< 0.001
Temperature (per °F)	−0.234	0.009	< 0.001
Wind speed	−0.105	0.095	0.266
Precipitation	0.104	0.027	< 0.001
Precipitation days	−0.007	0.011	0.538
Manufacturing enterprises	0.078	0.014	< 0.001
Population (thousands)	−0.053	0.014	< 0.001
Q2 (vs Q1)	−2.37	0.180	< 0.001
Q3 (vs Q1)	−2.44	0.189	< 0.001
Q4 (vs Q1)	−0.19	0.112	0.090

Variance. The variance between cities is τ² = 226.8 and the variance within a city is σ² = 0.75, so the intraclass correlation is 0.997: 99.7% of the variation left after the fixed effects is a difference between cities. The fixed effects alone explain 59.2% of all variation (marginal R^2 = 0.592); with the city baselines the model explains 99.9% (conditional R^2 = 0.999).

Interpretation. Once every city has its own baseline, a warmer quarter has lower SO2 (−0.234 per °F), and Q2 and Q3 are about 2.4 µg/m³ lower than Q1. This temperature effect is smaller than the −0.58 per °F in the regression, which also picks up differences between cities. The season effect has the opposite sign in the regression (where it is not significant), so we do not claim a season effect. Manufacturing and population must again be read together. Wind speed and precipitation days are not significant.

Using the city's own readings (k = 1, 2)

After k quarters the city's prediction is the Index plus a weighted share of the average miss on the revealed quarters (observed − Index):

$$\hat{y} = I + \lambda m, \qquad \lambda = \frac{k\tau^2}{k\tau^2 + \sigma^2}$$

where I is the SO2 Index (the fixed part), m is the average miss on the k revealed quarters, and λ is the weight given to it.

This is partial pooling (empirical Bayes). Because σ² is tiny next to τ², λ ≈ 0.997 after a single quarter: one quarter already pins down the city's baseline almost completely.

Table 8. Mixed-effects model, leave-one-city-out

Information about the new city	RMSE	R^2	95% CI for RMSE
k = 0 (none)	15.44	0.586	11.0 to 20.2
k = 1 (first quarter)	1.23	0.997	1.06 to 1.39
k = 2 (two quarters)	0.93	0.999	0.83 to 1.02

A prediction band for a city with no data

For a new city with no SO2 readings we use conformal prediction, an established distribution-free method. Each city gets one score, its worst quarterly miss, because the quarters of a city are not independent. With 41 cities and a 10% error rate the 38th smallest score gives the band Index ± 22.67 µg/m³, which contains all of a new city's quarters about 90% of the time, provided the new city is comparable to the 41 study cities (a large US city of 1969–1971). The band is wide next to an average SO2 of 30 µg/m³: it shows how rough a prediction for a city with no data is. The guarantee holds on average over new cities, not for every subgroup of cities.

Diagnostics

The residuals within cities show little autocorrelation (lag-1 correlation 0.04, Durbin-Watson 1.78) and only slight skew (0.15). The city intercepts have a heavy tail (Providence +48, Pittsburgh +34, Albany +21 above what size and industry predict), which is why we use distribution-free conformal bands and not normal-based intervals. Something not in the data, probably the type of industry or the fuel used, drives these cities. The Shapiro-Wilk tests on the residuals ($p = 0.016$) and the city intercepts ($p = 0.025$) show mild departures from normality. As a high/low classifier (above or below the median city of 26.0 µg/m³) the Index reaches AUC 0.66, which is modest.

5 COMPARING THE MODELS

All rows use the same protocol: leave-one-city-out on the same data, scoring only the quarters that were not revealed, with pooled R^2 measured against the mean of the other 40 cities.

Table 9. Final comparison (RMSE in µg/m³)

Model	k = 0 RMSE	k = 0 R^2	k = 0 95% CI	k = 1 RMSE	k = 2 RMSE
Benchmark (mean of other 40)	23.98	0	16.6 to 31.5	-	-
Multiple linear regression	16.03	0.553	11.7 to 20.6	2.75	2.09
Lasso	16.10	0.549	11.5 to 21.2	2.64	2.04
Ridge	16.39	0.533	11.6 to 21.8	3.23	2.55
Random forest	19.09	0.367	13.3 to 24.7	4.05	3.45
Mixed-effects	**15.44**	**0.586**	**11.0 to 20.2**	**1.23**	**0.93**

Intervals come from resampling whole cities 10,000 times.

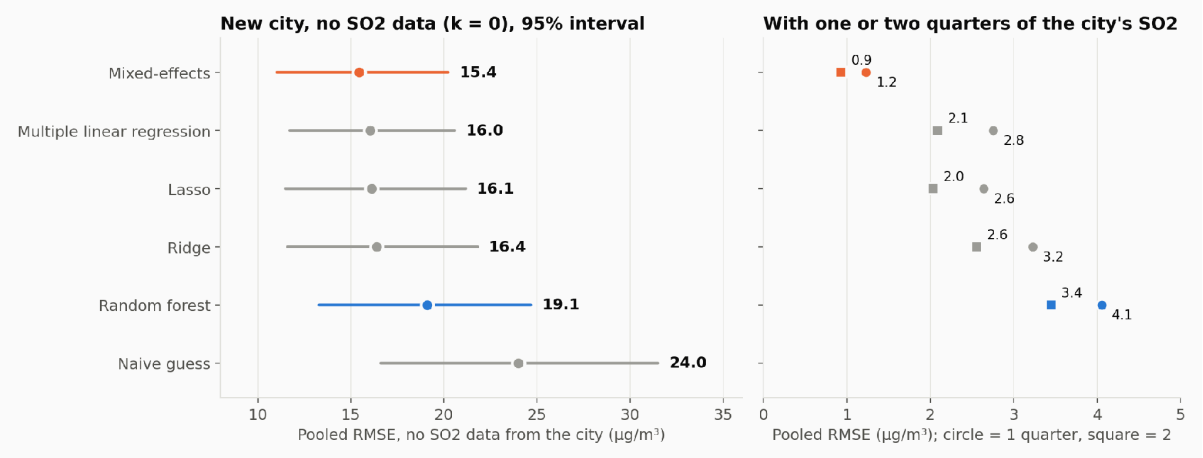


Figure 12. Left: error for a brand-new city (k = 0). Right: error once one or two quarters of that city's SO2 are known.

A brand-new city (k = 0). The four linear and mixed models are close together: RMSE between 15.4 and 16.4 against 23.98 for the benchmark. The mixed model is lowest, but its advantage over regression, Lasso and Ridge is only 0.6 to 1.0 and the 95% interval for each gap includes zero (for regression, −0.4 to 2.0). With 41 cities we cannot say these four differ when nothing is known about the city. The random forest is further behind (19.1, R^2 0.37); its gap to the mixed model is 3.7 with an interval of −0.04 to 8.1, so it is probably worse but not proven worse.

Once the city's own readings are known (k = 1, 2). The picture is clear. The mixed model reaches 1.23 and 0.93, regression, Lasso and Ridge 2.0 to 3.2, and the forest 3.4 to 4.1, and every gap to the mixed model has an interval above zero. The mixed model uses the revealed quarters best: it treats a city's baseline as its own random intercept, so one quarter almost pins it down (λ = 0.997), whereas the other models only shift their prediction.

6 USING THE MODEL ON A NEW CITY

To predict SO2 for a city that is not in our data, we recommend the mixed-effects model:

- **Step 1.** Collect the city's inputs: quarterly temperature, wind speed, precipitation and precipitation days, its number of manufacturing enterprises, its population, and which quarter it is.
- **Step 2.** Plug them into the fixed part of the fitted model (symbols as in the notation table in Section 4.2):

$$\hat{y} = 38.08 - 0.234\,T - 0.105\,W + 0.104\,R - 0.007\,D$$

$$+ 0.078\,M - 0.053\,P - 2.37\,Q_2 - 2.44\,Q_3 - 0.19\,Q_4$$

ŷ = predicted SO2 (µg/m³). T temperature (°F), W wind speed (mph), R precipitation (inches), D precipitation days, M manufacturing enterprises, P population (thousands). Q2, Q3, Q4 are 1 in that quarter and 0 otherwise (Q1 is the baseline).

- **Step 3.** Read the answer with its band: prediction ± 22.67 µg/m³ (90% band). If the SO2 of even one quarter can be measured, add 0.997 × (measured − predicted) to every other quarter.

Worked example: San Francisco

We took San Francisco, which was left out of the fit: its error (RMSE 10.7) equals the median across the 41 cities, so it is a typical case and not an easy one. With no San Francisco data the prediction (blue line in Figure 13) has the right seasonal shape, low in summer and high in winter, but the level is about 10 µg/m³ too high, while the actual values stay inside the 90% band in all 12 quarters. After seeing just the first quarter (orange line) the prediction sits almost on top of the actual values and the RMSE for the remaining quarters falls from 10.7 to 1.7.

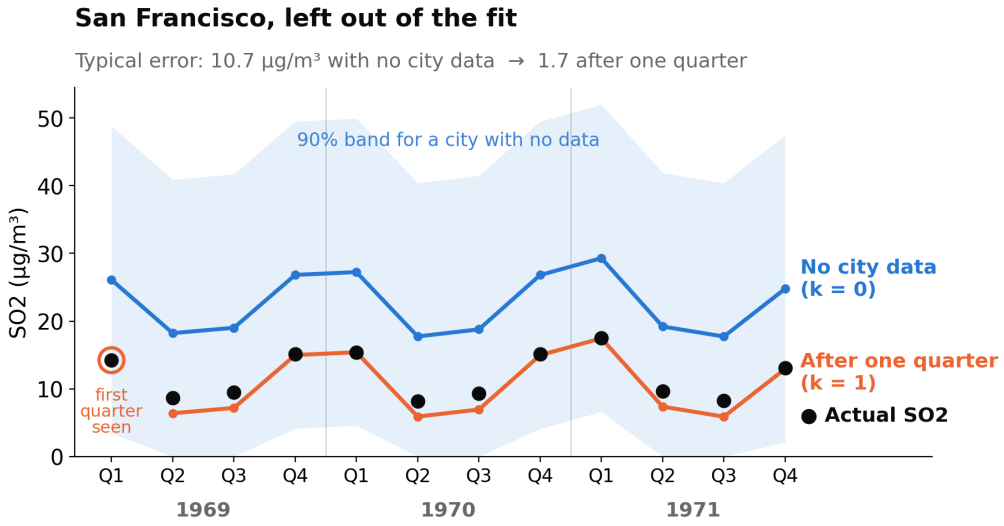


Figure 13. San Francisco: actual SO2 (black dots), prediction with no city data (blue, with 90% band) and after the first quarter is revealed (orange). Code: so2_worked_example.py.

7 CONCLUSION

From the exploration and the models we saw that city size and industry are the main drivers of SO2, that temperature is the clearest weather effect, and that most of the variation is between cities and not over time. We then tested four models on cities they had not seen. The assumptions of plain regression do not hold (the errors are skewed and almost all of the leftover error is a city offset), so we used clustered standard errors for inference and tested predictions on whole cities.

7.1 Answers to the research questions

Question: *RQ1: What drives SO2 in the 41 cities?*
Answer: City size and industry are the main driver. Manufacturing (+0.070 µg/m³ per enterprise) and population (−0.046 per thousand) are both significant with city-clustered errors ($p < 0.001$ and $p = 0.002$), but they are 96% correlated and must be read together: cities with many factories for their size have more SO2. Temperature is the clearest weather effect: colder conditions go with more SO2 (about −0.6 per °F across cities, $p = 0.010$, and −0.23 per °F once each city has its own baseline). Wind speed, precipitation and season are not reliable once the clustering is accounted for, and the season effect has different signs in different models, so we do not claim one. The random forest agrees: shuffling industry and size hurts its predictions most. All of these are associations, not causes.

Question: *RQ2: How well can SO2 be predicted for a new city?*
Answer: With no readings from the city, the best models explain about 55% to 59% of the variation (RMSE about 15 to 16 against 24 for guessing the average), so the prediction is useful as a rough screen but not precise: the 90% band is about ±23 µg/m³. With one quarter of readings the error falls to about 1.2 with the mixed-effects model, and with two quarters to about 0.9.

Question: *RQ3: Which of the four models predicts best?*
Answer: For a brand-new city the regression, Lasso, Ridge and mixed-effects models cannot be told apart with 41 cities (RMSE 15.4 to 16.4), and the random forest is weakest (19.1). Once one or two quarters of the city's readings are known, the mixed-effects model is clearly the best. We recommend the mixed-effects model, with plain regression as the simple alternative: for a completely new city regression is nearly as good, so the extra complexity only pays off once some of the city's own readings are available.

Overall answer. Yes, SO2 in a city that is not in our data can be predicted from its industry, size and weather, better than by guessing the average, but only roughly. A single measurement from the city is worth more than the choice of model.

7.2 Limitations

- There are only 41 cities, which is why the intervals are wide and why the random forest and the penalised models could not beat plain regression.
- Some cities (Providence, Pittsburgh, Chicago, St. Louis, Cleveland) are far above what their size and industry predict. Something not in the data, such as the type of industry or fuel, matters.
- Two cities (Memphis and Charleston) report impossible wind speeds; predictions are not affected but the wind-speed coefficient is unreliable. Some quarterly values come from interpolated months.

- Within a city SO2 hardly changes from quarter to quarter (intraclass correlation 0.997), so most of what we predict is the city's overall level.
- The data are from 1969 to 1971, and relationships are associations and not causes.

APPENDIX: CODE

All code is Python (pandas, numpy, statsmodels, scikit-learn). All models are scored with the shared script so2_eval.py, so the numbers in this report can be reproduced by running each file on quarterly_SO2_data.csv.

A1. so2_eval.py: shared leave-one-city-out scoring

```
"""
STTN327 SO2 project - SHARED evaluation (leave-one-city-out, k = 0, 1, 2)
Everyone scores their model with this, so the comparison table is fair.

Usage in your own notebook/script:

    from so2_eval import load_data, evaluate
    d = load_data("quarterly_SO2_data.csv")

    def my_model(train, test):
        # fit on `train` (40 cities), return predictions (numpy array) for every row of `test`
        ...
        return preds

    print(evaluate(my_model, d))          # -> table with k = 0, 1, 2

Protocol (from the group starter doc):
  k = 0 : predict the left-out city from its weather/season/manufacturing/population only.
  k = 1,2: reveal that city's first k quarters of SO2, add the average miss on those
           quarters to the remaining predictions, score ONLY the remaining quarters.
  Pooled RMSE, and pooled R2 = 1 - SSE(model) / SSE(mean SO2 of the other 40 cities).
Run `python so2_eval.py quarterly_SO2_data.csv` for examples (OLS, Ridge, Lasso, random forest).
"""
import sys, warnings
import numpy as np, pandas as pd
warnings.filterwarnings("ignore")

FEATURES = ["temperature", "windspeed", "precipitation", "precipitationdays",
            "manufacturing_enterprises", "population_thousands", "Q2", "Q3", "Q4"]

def load_data(path="quarterly_SO2_data.csv"):
    d = pd.read_csv(path)
    d["quarter"] = d["quarter"].astype(int)
    d = d.sort_values(["city", "year", "quarter"]).reset_index(drop=True)
    d["t"] = d.groupby("city").cumcount()          # 0..11 within city
    for q in (2, 3, 4):
        d[f"Q{q}"] = (d.quarter == q).astype(float)
    return d

def loco_predict(fit_predict, data, verbose=False):
    """Leave-one-city-out base predictions. Returns one row per quarter with columns base, naive."""
    out = []
    for c in data.city.unique():
        tr, te = data[data.city != c], data[data.city == c].copy()
        te["base"] = np.asarray(fit_predict(tr, te), dtype=float)
        te["naive"] = tr.SO2.mean()
        out.append(te)
        if verbose: print("done", c, flush=True)
    return pd.concat(out)

def calibrate(L, k, lam=1.0):
    """Add (lam x) the average miss on the first k quarters; keep only the remaining quarters."""
    sub = L.copy()
    if k > 0:
        miss = sub[sub.t < k].groupby("city").apply(lambda g: (g.SO2 - g.base).mean())
        sub["pred"] = sub.base + lam * sub.city.map(miss)
    else:
        sub["pred"] = sub.base
    return sub[sub.t >= k]

def score(L, ks=(0, 1, 2), shrink=None, data=None):
    rows = []
    for k in ks:
        lam = 1.0 if (shrink is None or k == 0) else shrink(k, data)
        sub = calibrate(L, k, lam)
        e, n = sub.SO2 - sub.pred, sub.SO2 - sub.naive
        per_city = (e**2).groupby(sub.city).mean().pow(0.5).mean()
        rows.append({"k": k, "pooled_RMSE": np.sqrt((e**2).mean()),
                     "pooled_R2": 1 - (e**2).sum() / (n**2).sum(), "avg_per_city_RMSE": per_city})
    return pd.DataFrame(rows).round(3).set_index("k")

def evaluate(fit_predict, data, ks=(0, 1, 2), shrink=None, verbose=False):
    """fit_predict(train_df, test_df) -> predictions for test_df rows.
    shrink(k, data) -> weight on the average miss (default 1.0 = add the full miss)."""
    return score(loco_predict(fit_predict, data, verbose), ks, shrink, data)

def naive_baseline(data):
    e = np.concatenate([d_c.SO2.values - data[data.city != c].SO2.mean()
                        for c, d_c in data.groupby("city")])
    return np.sqrt((e**2).mean())

# ------------------------------------------------------------------ examples
if __name__ == "__main__":
    from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
    from sklearn.preprocessing import StandardScaler
    from sklearn.pipeline import make_pipeline
    from sklearn.ensemble import RandomForestRegressor
    import statsmodels.formula.api as smf

    d = load_data(sys.argv[1] if len(sys.argv) > 1 else "quarterly_SO2_data.csv")
```

```
print(f"Naive baseline pooled RMSE (should be 23.98): {naive_baseline(d):.2f}\n")

def sk(model):
    return lambda tr, te: model.fit(tr[FEATURES], tr.SO2).predict(te[FEATURES])

def mixed(tr, te):
    m = smf.mixedlm("SO2 ~ " + " + ".join(FEATURES), tr, groups=tr.city).fit(reml=True)
    b = m.fe_params
    return b["Intercept"] + te[FEATURES].values @ b[FEATURES].values

def mixed_shrink(k, data):                          # empirical-Bayes weight used in the doc
    m = smf.mixedlm("SO2 ~ " + " + ".join(FEATURES), data, groups=data.city).fit(reml=True)
    t2, s2 = float(m.cov_re.iloc[0, 0]), float(m.scale)
    return t2 * k / (t2 * k + s2)

print("Mixed-effects (should give k=0 RMSE 15.44, R2 0.586):\n", evaluate(mixed, d, shrink=mixed_shrink), "\n")
print("Multiple linear regression (OLS):\n", evaluate(sk(LinearRegression()), d), "\n")
print("Ridge:\n", evaluate(sk(make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-2, 3, 30)))), d), "\n")
print("Lasso:\n", evaluate(sk(make_pipeline(StandardScaler(), LassoCV(cv=5, max_iter=20000))), d), "\n")
print("Random forest (300 trees):\n", evaluate(sk(RandomForestRegressor(300, random_state=1, n_jobs=-1)), d))
```

A2. Multiple linear regression (reviewed version)

```
# -*- coding: utf-8 -*-
"""REVIEWED COPY of Python Project(MLR) - original file left untouched.
Changes (model and leave-one-city-out evaluation are UNCHANGED and were verified):
  1. Added city-clustered standard errors (the 492 rows are 41 cities x 12 quarters, not independent).
  2. Rewrote the coefficient interpretation from the clustered results (no causal wording).
  3. Added the explanation of the predictor choice that the group starter doc asks for.
  4. Added robustness checks: without Memphis/Charleston windspeed, and without Chicago.
  5. Added one line on what the residual diagnostics mean.

Python Project(MLR).ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1ec-o9lS0aazj3DhjF0lcl5I9XtfD9bCP
"""

from google.colab import files
uploaded = files.upload()

import pandas as pd
import numpy as np
import statsmodels.api as sm

df = pd.read_csv('quarterly_SO2_data.csv')
print(df.head())
# 2. Create Quarter Dummies (Q1 is reference category)
df_dummies = pd.get_dummies(df.copy(), columns=['quarter'], drop_first=True, dtype=int)

# Predictor columns
X_cols = ['temperature', 'windspeed', 'precipitation', 'precipitationdays', 'manufacturing_enterprises',
'population_thousands',
    'quarter_2', 'quarter_3', 'quarter_4']
# Response variable
y_col = 'SO2'

# Predictor choice (we keep all nine, on purpose):
#  Weather: temperature, windspeed, precipitation, precipitationdays = the weather part of the research question.
#  Industry & size: manufacturing_enterprises, population_thousands = city-level pollution sources.
#  Season: quarter_2..quarter_4 (Q1 = baseline) = seasonal shifts such as heating.
#  We do NOT drop manufacturing or population even though r = 0.96 (VIF about 13): we want to predict, and
#  collinearity does not harm prediction, it only stops us reading the two slopes one at a time.
#  windspeed is kept so the model matches the other models, but its Memphis and Charleston values are
#  impossible (about 92 and 65 mph) - see the robustness check further down.

# Constant for intercept (beta_0)
X = sm.add_constant(df_dummies[X_cols])
y = df_dummies[y_col]

# Run regression
model = sm.OLS(y, X).fit()
print(model.summary())

# The 492 rows are 41 cities x 12 quarters, so rows from one city are strongly related (Durbin-Watson above
# is 0.15, far from 2). The ordinary standard errors above are therefore too small. Cluster by city instead:
city_id = df_dummies['city'].astype('category').cat.codes
model_cl = sm.OLS(y, X).fit(cov_type='cluster', cov_kwds={'groups': city_id})
compare = pd.DataFrame({'coef': model.params, 'se_ordinary': model.bse, 'se_clustered': model_cl.bse,
                        'p_ordinary': model.pvalues, 'p_clustered': model_cl.pvalues}).round(4)
print("\n=== Ordinary vs city-clustered standard errors (USE THE CLUSTERED p-VALUES) ===")
print(compare)
print("========MODEL COEF===============")
# Fitted model equation
coefs = model.params
print(coefs)
print("\n" + "=" * 50)
print("=== FITTED MODEL EQUATION ===")
print("=" * 50)
print(f"SO2_hat = {coefs['const']:.4f} "
      f"+ ({coefs['temperature']:.4f} * Temp) "
      f"+ ({coefs['windspeed']:.4f} * Wind) "
      f"+ ({coefs['precipitation']:.4f} * Precip) "
      f"+ ({coefs['precipitationdays']:.4f} * PrecDays) "
      f"+ ({coefs['manufacturing_enterprises']:.4f} * Manu) "
      f"+ ({coefs['population_thousands']:.4f} * Pop) "
      f"+ ({coefs['quarter_2']:.4f} * Q2) "
      f"+ ({coefs['quarter_3']:.4f} * Q3) "
      f"+ ({coefs['quarter_4']:.4f} * Q4)")

"""Interpreting the coefficients (p-values are from city-clustered standard errors, because the 492 rows are
only 41 independent cities; the ordinary p-values would overstate significance).

Intercept: 43.41 (p < 0.001) is the predicted SO2 in quarter 1 when every numerical predictor is zero. It is
outside the range of the data, so it is not interpretable on its own.

Temperature: -0.581 ug/m3 per degree F (p = 0.010). Significant. Cities and quarters that are 1 F warmer have about
0.58 lower SO2 on average, holding the other variables fixed. This is an association; it is consistent with more
heating fuel being burned in cold conditions, but this data cannot show the cause.

Manufacturing enterprises: +0.070 per enterprise (p < 0.001); Population: -0.046 per thousand people (p = 0.002).
Both significant, but the two are 96% correlated (VIF about 13), so read them together: cities with many factories
relative to their population have higher SO2. Neither slope should be interpreted alone.

Wind speed: -0.085 (p = 0.21) and precipitation: +0.278 (p = 0.20) are NOT significant once the clustering is
accounted for. Precipitation days (+0.123, p = 0.71) and the quarter indicators Q2, Q3, Q4 (p = 0.40, 0.38, 0.23)
are not significant either. (The ordinary p-values of 0.049 for wind speed and < 0.001 for precipitation came from
treating the rows as independent and should not be quoted.) The wind speed slope is also distorted: Memphis (about
92 mph) and Charleston (about 65 mph) report impossible values in every quarter. Without those two cities the slope
is -2.6 per mph (clustered p = 0.10), so there is some sign of a negative wind effect, but the data is too thin to
claim it.

Model fit and predictive performance: R-squared is 0.64 on the 41 cities we have, but the fair figure for a NEW city
is the leave-one-city-out result at the bottom: pooled R-squared 0.553, RMSE 16.0 against 24.0 for the naive guess
```

```
(k = 0), and RMSE 2.75 and 2.09 after one and two quarters of readings. So the model predicts a new city moderately
well, not accurately. Collinearity between manufacturing and population does not hurt this prediction; it only makes
the two individual slopes hard to read.

Assumption checks: the residuals are right-skewed (skew 0.87, Shapiro-Wilk p < 0.001) and their spread is not
constant (Breusch-Pagan p < 0.001), so a log transform of SO2 would be the natural next step; and the residuals are
almost entirely city-level (98% of residual variance is between cities), which is why the clustered standard errors
are the correct ones. Chicago has a strong pull on the fit: R-squared falls from 0.64 to 0.49 without it.
"""

import matplotlib.pyplot as plt
from scipy import stats
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import het_breuschpagan

resid = model.resid
fitted = model.fittedvalues

fig, ax = plt.subplots(2, 2, figsize=(12, 9))
ax[0,0].scatter(fitted, resid, alpha=.6); ax[0,0].axhline(0, color='r')
ax[0,0].set(title='Resi vs fit', xlabel='Fit SO2', ylabel='Resid')
sm.qqplot(resid, line='s', ax=ax[0,1]); ax[0,1].set_title('Normal QQ plot')
ax[1,0].hist(resid, bins=30, edgecolor='k'); ax[1,0].set_title('Resid histogram')
df_dummies['resid'] = resid
order = df_dummies.groupby('city')['resid'].mean().sort_values().index
ax[1,1].boxplot([df_dummies.loc[df_dummies.city == c, 'resid'] for c in order])
ax[1,1].axhline(0, color='r'); ax[1,1].set_xticks([])
ax[1,1].set_title('Resid by city')
plt.tight_layout(); plt.show()

print("Shapiro-Wilk:", stats.shapiro(resid))
print("Skew:", stats.skew(resid), " Excess kurtosis:", stats.kurtosis(resid))
print("Breusch-Pagan p-value:", het_breuschpagan(resid, X)[1])

vif = pd.Series([variance_inflation_factor(X.values, i) for i in range(1, X.shape[1])],
                index=X.columns[1:])
print(vif.round(1))
print("corr(manu, pop):", df['manufacturing_enterprises'].corr(df['population_thousands']))
print("Share of residual variance between cities:",
      (df_dummies.groupby('city')['resid'].mean()**2).mean() / (resid**2).mean())

# --- Robustness checks (they support the interpretation text above) ---
def clustered(sub):
    Xs = sm.add_constant(sub[X_cols]); cid = sub['city'].astype('category').cat.codes
    return sm.OLS(sub['SO2'], Xs).fit(cov_type='cluster', cov_kwds={'groups': cid})
no_wind_err = clustered(df_dummies[~df_dummies.city.isin(['Memphis', 'Charleston'])])
print("\nWithout Memphis and Charleston: windspeed coef %.3f, clustered p = %.3f" %
      (no_wind_err.params['windspeed'], no_wind_err.pvalues['windspeed']))
no_chi = sm.OLS(df_dummies.loc[df_dummies.city != 'Chicago', 'SO2'],
                sm.add_constant(df_dummies.loc[df_dummies.city != 'Chicago', X_cols])).fit()
print("R-squared with Chicago %.3f, without Chicago %.3f" % (model.rsquared, no_chi.rsquared))

df = df.sort_values(['city', 'year', 'quarter']).reset_index(drop=True)
d  = pd.get_dummies(df, columns=['quarter'], drop_first=True, dtype=int)

def loco(k):
    y_true, y_pred, y_naive = [], [], []
    for c in d['city'].unique():
        train, test = d[d.city != c], d[d.city == c]
        m = sm.OLS(train['SO2'], sm.add_constant(train[X_cols])).fit()
        pred = m.predict(sm.add_constant(test[X_cols], has_constant='add')).values
        obs  = test['SO2'].values
        if k > 0:
            pred = pred + (obs[:k] - pred[:k]).mean()
        y_true.append(obs[k:]); y_pred.append(pred[k:])
        y_naive.append(np.full(12 - k, train['SO2'].mean()))
    y_true, y_pred, y_naive = map(np.concatenate, (y_true, y_pred, y_naive))
    rmse = np.sqrt(np.mean((y_true - y_pred)**2))
    r2   = 1 - np.sum((y_true - y_pred)**2) / np.sum((y_true - y_naive)**2)
    return rmse, r2

results = pd.DataFrame([(k, *loco(k)) for k in (0, 1, 2)],
                       columns=['k', 'Pooled RMSE', 'Pooled R2']).round(3)
print(results)
```

A3. Ridge and Lasso (reviewed version)

```
"""REVIEWED COPY of Ridge_Lasso.py - original file left untouched.
The leave-one-city-out evaluation (k = 0, 1, 2) is UNCHANGED. Changes:
  1. The penalty (alpha) is now chosen with cross-validation split BY CITY inside each training set
     (the original used row-wise splits, which let quarters of the same city sit in both parts).
  2. The alpha grid now reaches lower (1e-3); Lasso kept hitting the bottom of the old grid (0.01).
  3. The chosen alphas are printed.
  4. Added a coefficient comparison against ordinary regression (a checklist item in the starter doc).
  5. Added the interpretation text.
"""
import os
import warnings
import numpy as np
import pandas as pd
from sklearn.linear_model import LassoCV, RidgeCV, LinearRegression
from sklearn.model_selection import GroupKFold
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings('ignore')


def evaluate_penalized_models(df, features, target):
    cities = df['city'].unique()

    # Dictionaries to store sum of squared errors (SSE) and total sum of squares (SST)
    results = {
        'Ridge': {
            'k0_sq_err': [],
            'k0_sq_tot': [],
            'k1_sq_err': [],
            'k1_sq_tot': [],
            'k2_sq_err': [],
            'k2_sq_tot': [],
        },
        'Lasso': {
            'k0_sq_err': [],
            'k0_sq_tot': [],
            'k1_sq_err': [],
            'k1_sq_tot': [],
            'k2_sq_err': [],
            'k2_sq_tot': [],
        },
    }

    ALPHAS = np.logspace(-3, 3, 100)
    chosen_alpha = {'Ridge': [], 'Lasso': []}

    # Leave-one-city-out cross-validation
    for city in cities:
        # Split train (40 cities) and test (1 left-out city)
        train = df[df['city'] != city]
        test = df[df['city'] == city].sort_values(['year', 'quarter'])

        X_train, y_train = train[features], train[target]
        X_test, y_test = test[features], test[target].values

        # Standardize features (crucial for Ridge/Lasso) based on training data
        scaler = StandardScaler()
        X_train_sc = scaler.fit_transform(X_train)
        X_test_sc = scaler.transform(X_test)

        # Baseline guess: Mean SO2 of the other 40 cities
        baseline_pred = y_train.mean()

        # Penalty chosen by 5-fold cross-validation split BY CITY, using the 40 training cities only
        city_folds = list(GroupKFold(n_splits=5).split(X_train_sc, y_train, groups=train['city']))
        models = {
            'Ridge': RidgeCV(alphas=ALPHAS, cv=city_folds, scoring='neg_mean_squared_error'),
            'Lasso': LassoCV(alphas=ALPHAS, cv=city_folds, max_iter=10000),
        }

        for name, model in models.items():
            # Fit model on the 40 cities
            model.fit(X_train_sc, y_train)
            chosen_alpha[name].append(model.alpha_)

            # Predict the 12 quarters of the held-out city
            preds = model.predict(X_test_sc)

            # --- k = 0: Zero information ---
            results[name]['k0_sq_err'].extend((y_test - preds) ** 2)
            results[name]['k0_sq_tot'].extend((y_test - baseline_pred) ** 2)

            # --- k = 1: Reveal first quarter ---
            miss_k1 = y_test[0] - preds[0]
            preds_k1 = preds[1:] + miss_k1  # Apply offset to remaining 11 quarters
            results[name]['k1_sq_err'].extend((y_test[1:] - preds_k1) ** 2)
            results[name]['k1_sq_tot'].extend((y_test[1:] - baseline_pred) ** 2)

            # --- k = 2: Reveal first two quarters ---
            miss_k2 = np.mean(y_test[:2] - preds[:2])
            preds_k2 = preds[2:] + miss_k2  # Apply offset to remaining 10 quarters
            results[name]['k2_sq_err'].extend((y_test[2:] - preds_k2) ** 2)
            results[name]['k2_sq_tot'].extend((y_test[2:] - baseline_pred) ** 2)

    # 3. Calculate and print pooled metrics
    print(f"{'Model':<10} | {'k':<2} | {'Pooled RMSE':<12} | {'Pooled R²':<10}")
    print("-" * 50)

    for name in ['Ridge', 'Lasso']:
        for k in [0, 1, 2]:
            sq_err = np.sum(results[name][f'k{k}_sq_err'])
            sq_tot = np.sum(results[name][f'k{k}_sq_tot'])
            n = len(results[name][f'k{k}_sq_err'])
```

```
            rmse = np.sqrt(sq_err / n)
            r2 = 1 - (sq_err / sq_tot)

            print(f"{name:<10} | {k:<2} | {rmse:<12.2f} | {r2:<10.3f}")

    print("\nChosen alpha over the 41 leave-one-city-out fits (features are standardised):")
    for name, a in chosen_alpha.items():
        print(f"  {name}: median {np.median(a):.3f}, min {np.min(a):.3f}, max {np.max(a):.3f}")
    return chosen_alpha


def compare_coefficients(df, features, target):
    """Fit ordinary regression, Ridge and Lasso on all 41 cities and compare the coefficients (original units)."""
    ALPHAS = np.logspace(-3, 3, 100)
    scaler = StandardScaler()
    Xs = scaler.fit_transform(df[features])
    folds = list(GroupKFold(n_splits=5).split(Xs, df[target], groups=df['city']))
    ols = LinearRegression().fit(df[features], df[target])
    ridge = RidgeCV(alphas=ALPHAS, cv=folds, scoring='neg_mean_squared_error').fit(Xs, df[target])
    lasso = LassoCV(alphas=ALPHAS, cv=folds, max_iter=10000).fit(Xs, df[target])
    table = pd.DataFrame({'OLS': ols.coef_,
                          'Ridge': ridge.coef_ / scaler.scale_,
                          'Lasso': lasso.coef_ / scaler.scale_}, index=features).round(4)
    print(f"\nCoefficients on all 41 cities (original units). Ridge alpha = {ridge.alpha_:.3f}, Lasso alpha =
{lasso.alpha_:.3f}")
    print(table)
    print("Predictors set to zero by Lasso:", [f for f, c in zip(features, lasso.coef_) if c == 0] or "none")


INTERPRETATION = """
What the penalty does. Ordinary regression picks the coefficients that fit the training data best. Ridge and Lasso add
a penalty that pulls the coefficients towards zero, controlled by alpha: Ridge shrinks all coefficients smoothly and
shares weight between correlated predictors; Lasso can shrink a coefficient exactly to zero, which removes the
predictor. Either can help when there are many predictors, correlated predictors (manufacturing and population have
r = 0.96) or little data (only 41 independent cities).

Choosing alpha. Alpha was chosen by 5-fold cross-validation inside the 40 training cities of every leave-one-city-out
fit, with the folds split by city, so the held-out city never influences the choice and quarters of the same city never
sit on both sides of a split. Lasso's chosen alpha is stable (median 0.2 on standardised predictors). Ridge's chosen
alpha jumps around between folds (median 11, from about 0 to 46), because the cross-validation curve is almost flat:
many alpha values are about equally good.

Coefficients (all 41 cities, original units). Lasso sets the Q4 indicator to exactly zero and roughly halves Q2 and Q3,
which fits the finding that season adds little. Ridge shrinks manufacturing (0.070 to 0.048) and population (-0.046
to -0.025) together, as expected for a correlated pair; the other coefficients move only a little. The signs and
ordering of the important effects (temperature negative, manufacturing positive, population negative) do not change.

Performance for a new city (leave-one-city-out). Ordinary regression gives RMSE 16.03 / R-squared 0.553 at k = 0.
Lasso (16.10 / 0.549) and Ridge (16.39 / 0.533) do not beat it; at k = 1 and 2 Lasso is marginally better
(2.64 and 2.04 against 2.75 and 2.09) and Ridge is worse. These differences are tiny compared with the uncertainty
that comes from having only 41 cities (a 95% interval for the k = 0 RMSE is roughly 12 to 20), so the honest
conclusion is that shrinkage does not improve prediction here. With nine predictors and 41 cities, the limit is the
number of cities, not the number of predictors.

Note: windspeed is kept, as in the other models, although its Memphis and Charleston values are impossible
(about 92 and 65 mph).
"""


def main():
    # 1. Load data and prepare features
    script_dir = os.path.dirname(os.path.abspath(__file__))
    data_path = os.path.join(script_dir, 'quarterly_SO2_data.csv')
    if not os.path.exists(data_path):
        data_path = 'quarterly_SO2_data.csv'

    df = pd.read_csv(data_path)

    # Create Quarter dummies (Q1 is the reference)
    df['Q2'] = (df['quarter'] == 2).astype(int)
    df['Q3'] = (df['quarter'] == 3).astype(int)
    df['Q4'] = (df['quarter'] == 4).astype(int)

    features = [
        'temperature',
        'windspeed',
        'precipitation',
        'precipitationdays',
        'manufacturing_enterprises',
        'population_thousands',
        'Q2',
        'Q3',
        'Q4',
    ]
    target = 'SO2'

    # 2. Run evaluation
    evaluate_penalized_models(df, features, target)
    compare_coefficients(df, features, target)


if __name__ == '__main__':
    main()
```

A4. so2_random_forest.py

```
"""
STTN327 SO2 project - RANDOM FOREST (new-city prediction)
=========================================================
Run:  python so2_random_forest.py quarterly_SO2_data.csv rf_output
Needs: pandas numpy scikit-learn statsmodels scipy matplotlib   (+ so2_eval.py in the same folder)

What this script does (each part prints its own results and saves figures/CSVs to rf_output/):
  1. Why leave-one-city-out?  OOB / random-row CV / leave-one-city-out R2 for the same forest.
  2. Baseline forest (default settings), with and without windspeed (the Memphis/Charleston check).
  3. NESTED tuning: hyper-parameters are chosen by city-grouped cross-validation INSIDE the
     40 training cities of every leave-one-city-out fold, so the held-out city is never used to tune.
  4. Final scores for k = 0, 1, 2 with the shared protocol (so2_eval.py).
  5. Comparison with reference runs of the other models + city-level bootstrap confidence intervals.
  6. Variable importance: grouped permutation importance (handles manufacturing/population r = 0.96).
  7. Partial dependence and a failure analysis (which cities does the forest get wrong, and why).
"""
import sys, os, time, pickle, warnings
import numpy as np, pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GroupKFold, KFold
from sklearn.inspection import partial_dependence
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
import statsmodels.formula.api as smf
from so2_eval import load_data, loco_predict, calibrate, score, FEATURES

warnings.filterwarnings("ignore")
CSV = sys.argv[1] if len(sys.argv) > 1 else "quarterly_SO2_data.csv"
OUT = sys.argv[2] if len(sys.argv) > 2 else "rf_output"
os.makedirs(OUT, exist_ok=True)
SEED = 2026
RF_FEATURES = [f for f in FEATURES if f != "windspeed"]   # windspeed dropped: Memphis/Charleston values are impossible
GRID = [(leaf, mtry) for leaf in (1, 3, 5, 10, 20) for mtry in (2, 3, 5)]   # (min_samples_leaf, max_features)
INNER_TREES, FINAL_TREES = 100, 500
CACHE = os.path.join(OUT, "nested_cache.pkl")

d = load_data(CSV)
cities = list(d.city.unique())
t0 = time.time()
def log(*a): print(f"[{time.time()-t0:6.0f}s]", *a, flush=True)

def rf(**kw):
    return RandomForestRegressor(random_state=SEED, n_jobs=-1, **kw)

# ------------------------------------------------------------------- plot style (dataviz skill)
SURFACE, INK, INK2, GRIDC = "#fcfcfb", "#0b0b0b", "#52514e", "#e5e4e0"
BLUE, ORANGE, AQUA, MUTED = "#2a78d6", "#eb6834", "#1baf7a", "#9c9b96"
plt.rcParams.update({"figure.facecolor": SURFACE, "axes.facecolor": SURFACE, "savefig.facecolor": SURFACE,
                     "text.color": INK, "axes.labelcolor": INK2, "xtick.color": INK2, "ytick.color": INK2,
                     "axes.edgecolor": GRIDC, "axes.grid": True, "grid.color": GRIDC, "grid.linewidth": 0.6,
                     "axes.spines.top": False, "axes.spines.right": False, "axes.axisbelow": True,
                     "font.size": 11, "axes.titlesize": 13, "axes.titleweight": "bold", "axes.titlelocation": "left"})
def save(fig, name):
    fig.tight_layout(); fig.savefig(os.path.join(OUT, name), dpi=200); plt.close(fig); log("saved", name)

# ================================================================ 1. why leave-one-city-out
log("1. leakage demonstration")
X, y, g = d[RF_FEATURES].values, d.SO2.values, d.city.values
m_oob = rf(n_estimators=500, oob_score=True).fit(X, y)
r2_oob = m_oob.oob_score_
# random 5-fold by ROW: quarters of the same city land in train and test
e_row, e_nv_row = [], []
for tr, te in KFold(5, shuffle=True, random_state=SEED).split(X):
    m = rf(n_estimators=300).fit(X[tr], y[tr])
    e_row += list(y[te] - m.predict(X[te])); e_nv_row += list(y[te] - y[tr].mean())
r2_row = 1 - np.sum(np.square(e_row)) / np.sum(np.square(e_nv_row))
L_def = loco_predict(lambda tr, te: rf(n_estimators=500).fit(tr[RF_FEATURES], tr.SO2).predict(te[RF_FEATURES]), d)
r2_loco_default = score(L_def, ks=(0,)).pooled_R2.iloc[0]
print(f"   Out-of-bag R2 (rows)      : {r2_oob:.3f}\n   Random 5-fold by row R2   : {r2_row:.3f}\n   Leave-one-city-out R2
: {r2_loco_default:.3f}")


# ================================================================ 2. baseline + windspeed sensitivity
log("2. baseline forests")
base_rows = []
for label, cols in (("RF default, with windspeed", FEATURES), ("RF default, no windspeed", RF_FEATURES)):
    Lb = loco_predict(lambda tr, te, c=cols: rf(n_estimators=500).fit(tr[c], tr.SO2).predict(te[c]), d)
    s = score(Lb); s.insert(0, "model", label); base_rows.append(s.reset_index())
baseline = pd.concat(base_rows)
print(baseline.to_string(index=False))

# ================================================================ 3. nested tuning
def select_params(train, return_scores=False):
    """Choose (min_samples_leaf, max_features) by city-grouped 5-fold CV inside `train` only."""
    Xt, yt, gt = train[RF_FEATURES].values, train.SO2.values, train.city.values
    folds = list(GroupKFold(5).split(Xt, yt, gt))
    scores = {}
    for leaf, mtry in GRID:
        e2 = []
        for tr_i, va_i in folds:
            m = rf(n_estimators=INNER_TREES, min_samples_leaf=leaf, max_features=mtry).fit(Xt[tr_i], yt[tr_i])
            e2 += list((yt[va_i] - m.predict(Xt[va_i])) ** 2)
        scores[(leaf, mtry)] = float(np.sqrt(np.mean(e2)))
    best = min(scores, key=lambda k: (round(scores[k], 6), -k[0]))          # tie-break: simpler (bigger leaf)
    return (best, scores) if return_scores else best

if os.path.exists(CACHE):
    log("3. loading cached nested-CV run", CACHE)
    L_tuned, chosen = pickle.load(open(CACHE, "rb"))
```

```
else:
    log("3. nested tuning: 41 outer folds x", len(GRID), "settings x 5 inner folds (about 10 minutes)")
    chosen = []
    def tuned_fit_predict(tr, te):
        leaf, mtry = select_params(tr)
        chosen.append({"held_out_city": te.city.iloc[0], "min_samples_leaf": leaf, "max_features": mtry})
        log(f"   {te.city.iloc[0]:24s} chose leaf={leaf:2d}, max_features={mtry}")
        return rf(n_estimators=FINAL_TREES, min_samples_leaf=leaf, max_features=mtry).fit(tr[RF_FEATURES],
tr.SO2).predict(te[RF_FEATURES])
    L_tuned = loco_predict(tuned_fit_predict, d)
    pickle.dump((L_tuned, chosen), open(CACHE, "wb"))
chosen = pd.DataFrame(chosen)
chosen.to_csv(os.path.join(OUT, "rf_nested_choices.csv"), index=False)
freq = chosen.groupby(["min_samples_leaf", "max_features"]).size().sort_values(ascending=False)
print("   Settings chosen across the 41 outer folds:\n", freq.to_string())

(best, grid_scores) = select_params(d, return_scores=True)          # final settings, tuned on all 41 cities
FINAL = dict(min_samples_leaf=best[0], max_features=best[1])
print(f"   FINAL settings (grouped 5-fold CV on all 41 cities): {FINAL}  inner CV RMSE = {grid_scores[best]:.2f}")
pd.DataFrame([{"min_samples_leaf": k[0], "max_features": k[1], "inner_cv_rmse": v} for k, v in grid_scores.items()]
             ).to_csv(os.path.join(OUT, "rf_tuning_grid.csv"), index=False)

leaves = sorted({k[0] for k in GRID}); mtrys = sorted({k[1] for k in GRID})
M = np.array([[grid_scores[(l, m)] for m in mtrys] for l in leaves])
fig, ax = plt.subplots(figsize=(7.2, 4.8))
im = ax.imshow(M, cmap="Blues_r", aspect="auto")
ax.set_xticks(range(len(mtrys))); ax.set_xticklabels(mtrys); ax.set_yticks(range(len(leaves))); ax.set_yticklabels(leaves)
ax.set_xlabel("max_features (features tried per split)"); ax.set_ylabel("min_samples_leaf"); ax.grid(False)
for i in range(len(leaves)):
    for j in range(len(mtrys)):
        ax.text(j, i, f"{M[i, j]:.1f}", ha="center", va="center", color="white" if M[i, j] < np.median(M) else INK,
                fontweight="bold" if (leaves[i], mtrys[j]) == best else "normal")
ax.set_title("Tuning grid: city-grouped CV RMSE (darker = better)")
save(fig, "fig2_tuning_grid.png")

# ================================================================ 4. final scores
log("4. final scores")
tuned = score(L_tuned); tuned.insert(0, "model", "RF tuned (nested CV), no windspeed")
naive_row = {"model": "Naive guess (mean of other 40 cities)", "k": 0, "pooled_RMSE":
round(float(np.sqrt(np.mean((L_tuned.SO2 - L_tuned.naive) ** 2))), 3), "pooled_R2": 0.0, "avg_per_city_RMSE": np.nan}
results = pd.concat([pd.DataFrame([naive_row]), baseline, tuned.reset_index()])
results.to_csv(os.path.join(OUT, "rf_results_table.csv"), index=False)
print(results.to_string(index=False))

fig, ax = plt.subplots(figsize=(8, 4.8))
labels = ["Out-of-bag\n(rows)", "Random 5-fold\n(rows)", "Leave-one-city-out\n(default forest)", "Leave-one-city-out\n(tuned,
nested)"]
vals = [r2_oob, r2_row, r2_loco_default, tuned.loc[0].pooled_R2]
bars = ax.bar(labels, vals, color=[MUTED, MUTED, BLUE, BLUE], width=0.55)
for b, v in zip(bars, vals): ax.text(b.get_x() + b.get_width() / 2, max(v, 0) + 0.02, f"{v:.2f}", ha="center", color=INK,
fontweight="bold")
ax.set_ylim(min(0, min(vals)) - 0.05, 1.08); ax.set_ylabel("R² (vs naive guess)")
ax.set_title("Same forest, very different score: only the new-city scores are honest"); ax.grid(axis="x", visible=False)
save(fig, "fig1_why_leave_one_city_out.png")

# ================================================================ 5. comparison + bootstrap
log("5. reference runs of the other models + city bootstrap")
def mixed_fp(tr, te):
    m = smf.mixedlm("SO2 ~ " + " + ".join(FEATURES), tr, groups=tr.city).fit(reml=True)
    b = m.fe_params; return b["Intercept"] + te[FEATURES].values @ b[FEATURES].values
mm = smf.mixedlm("SO2 ~ " + " + ".join(FEATURES), d, groups=d.city).fit(reml=True)
t2, s2 = float(mm.cov_re.iloc[0, 0]), float(mm.scale)
mixed_lam = lambda k, data: t2 * k / (t2 * k + s2)
def sk(model): return lambda tr, te: model.fit(tr[FEATURES], tr.SO2).predict(te[FEATURES])
refs = {"Mixed-effects": (loco_predict(mixed_fp, d), mixed_lam),
        "Linear regression": (loco_predict(sk(LinearRegression()), d), None),
        "Ridge": (loco_predict(sk(make_pipeline(StandardScaler(), RidgeCV(alphas=np.logspace(-2, 3, 30)))), d), None),
        "Lasso": (loco_predict(sk(make_pipeline(StandardScaler(), LassoCV(cv=5, max_iter=20000))), d), None),
        "Random forest (tuned)": (L_tuned, None)}
KS = (0, 1, 2)
def city_stats(L, k, lam):
    sub = calibrate(L, k, 1.0 if k == 0 else (lam(k, d) if lam else 1.0))
    e2 = (sub.SO2 - sub.pred) ** 2; n2 = (sub.SO2 - sub.naive) ** 2
    return (e2.groupby(sub.city).sum().reindex(cities).values, n2.groupby(sub.city).sum().reindex(cities).values,
            sub.groupby("city").size().reindex(cities).values)
rng = np.random.default_rng(SEED); B = 5000
IDX = rng.integers(0, len(cities), (B, len(cities)))                # resample whole cities (the independent units)
boot, point = {}, {}
for name, (L, lam) in refs.items():
    for k in KS:
        sse, sst, n = city_stats(L, k, lam)
        rm = np.sqrt(sse[IDX].sum(1) / n[IDX].sum(1)); r2 = 1 - sse[IDX].sum(1) / sst[IDX].sum(1)
        boot[(name, k)] = (rm, r2); point[(name, k)] = (np.sqrt(sse.sum() / n.sum()), 1 - sse.sum() / sst.sum())
sse0, sst0, n0 = city_stats(L_tuned, 0, None)
naive_boot = np.sqrt(sst0[IDX].sum(1) / n0[IDX].sum(1)); naive_pt = np.sqrt(sst0.sum() / n0.sum())
rows = []
for (name, k), (rm, r2) in boot.items():
    rows.append({"model": name, "k": k, "RMSE": point[(name, k)][0], "RMSE_lo": np.percentile(rm, 2.5), "RMSE_hi":
np.percentile(rm, 97.5),
                 "R2": point[(name, k)][1], "R2_lo": np.percentile(r2, 2.5), "R2_hi": np.percentile(r2, 97.5)})
comp = pd.DataFrame(rows).round(3); comp.to_csv(os.path.join(OUT, "rf_model_comparison_bootstrap.csv"), index=False)
print("   (reference re-implementations; the group's own model numbers go in the final table)")
print(comp.to_string(index=False))
print("\n   Paired difference in pooled RMSE, RF minus other model (positive = RF worse); 95% city-bootstrap CI")
diffs = []
for other in ("Mixed-effects", "Linear regression", "Ridge", "Lasso"):
    for k in KS:
        dlt = boot[("Random forest (tuned)", k)][0] - boot[(other, k)][0]
        diffs.append({"vs": other, "k": k, "RMSE_diff": point[("Random forest (tuned)", k)][0] - point[(other, k)][0],
                      "lo": np.percentile(dlt, 2.5), "hi": np.percentile(dlt, 97.5), "P(RF worse)": float((dlt > 0).mean())})
diffs = pd.DataFrame(diffs).round(3); diffs.to_csv(os.path.join(OUT, "rf_paired_differences.csv"), index=False)
print(diffs.to_string(index=False))

# figure 6: k = 0 pooled RMSE with 95% CI
order = [("Naive guess", naive_pt, np.percentile(naive_boot, 2.5), np.percentile(naive_boot, 97.5))]
for name in refs:
```

```
    c = comp[(comp.model == name) & (comp.k == 0)].iloc[0]; order.append((name, c.RMSE, c.RMSE_lo, c.RMSE_hi))
order.sort(key=lambda r: r[1])
fig, ax = plt.subplots(figsize=(8.4, 4.8))
for i, (name, v, lo, hi) in enumerate(order):
    col = BLUE if name.startswith("Random") else (ORANGE if name.startswith("Mixed") else MUTED)
    ax.plot([lo, hi], [i, i], color=col, lw=2, solid_capstyle="round"); ax.plot(v, i, "o", color=col, ms=9, mec=SURFACE,
mew=2)
    ax.text(hi + 0.5, i, f"{v:.1f}", va="center", color=INK, fontweight="bold")
ax.set_yticks(range(len(order))); ax.set_yticklabels([o[0] for o in order]); ax.invert_yaxis()
ax.set_xlabel("Pooled RMSE for a brand-new city (µg/m³), 95% CI from resampling cities"); ax.grid(axis="y", visible=False)
ax.set_title("Predicting SO2 in a new city (k = 0): lower is better"); ax.set_xlim(8, order[-1][3] + 4)
save(fig, "fig6_model_comparison_k0.png")

# ================================================================ 6. permutation importance (grouped)
log("6. permutation importance")
BLOCKS = {"City size & industry (manufacturing + population)": ["manufacturing_enterprises", "population_thousands"],
          "  manufacturing alone": ["manufacturing_enterprises"], "  population alone": ["population_thousands"],
          "Temperature": ["temperature"], "Precipitation (amount + days)": ["precipitation", "precipitationdays"],
          "Season (quarter)": ["Q2", "Q3", "Q4"]}
prng = np.random.default_rng(SEED); NREP = 30
imp = {k: [] for k in BLOCKS}
for tr_i, te_i in GroupKFold(10).split(X, y, g):
    m = rf(n_estimators=300, **FINAL).fit(X[tr_i], y[tr_i])
    Xte, yte = X[te_i].copy(), y[te_i]
    base = np.sqrt(np.mean((yte - m.predict(Xte)) ** 2))
    for name, cols in BLOCKS.items():
        ci = [RF_FEATURES.index(c) for c in cols]; inc = []
        for _ in range(NREP):
            Xp = Xte.copy(); perm = prng.permutation(len(Xp)); Xp[:, ci] = Xte[perm][:, ci]     # permute the block jointly
            inc.append(np.sqrt(np.mean((yte - m.predict(Xp)) ** 2)) - base)
        imp[name].append(np.mean(inc))
imp_df = pd.DataFrame([{"feature_or_block": k.strip(), "rmse_increase_mean": np.mean(v), "rmse_increase_sd_over_folds":
np.std(v, ddof=1)} for k, v in imp.items()]).round(3)
m_all = rf(n_estimators=FINAL_TREES, **FINAL).fit(X, y)
imp_df.to_csv(os.path.join(OUT, "rf_permutation_importance.csv"), index=False)
print(imp_df.to_string(index=False))
print("   Impurity importance (all 41 cities; biased towards high-variance features):")
print(pd.Series(m_all.feature_importances_, RF_FEATURES).sort_values(ascending=False).round(3).to_string())
sel = imp_df.sort_values("rmse_increase_mean")
fig, ax = plt.subplots(figsize=(9, 4.8))
colors = [BLUE if "City size" in n or n in ("manufacturing alone", "population alone") else MUTED for n in
sel.feature_or_block]
ax.barh([n.replace(" (manufacturing + population)", "\n(manufacturing + population)") for n in sel.feature_or_block],
sel.rmse_increase_mean, xerr=sel.rmse_increase_sd_over_folds, color=colors, height=0.55,
        error_kw=dict(ecolor=INK2, lw=1, capsize=3))
for i, v in enumerate(sel.rmse_increase_mean): ax.text(max(v, 0) + 0.6, i + 0.28, f"{max(v, 0):+.1f}" if abs(v) >= 0.05 else
"0.0", color=INK, fontsize=10, fontweight="bold", va="center")
ax.set_xlabel("Increase in held-out RMSE (µg/m³) when the feature is shuffled"); ax.grid(axis="y", visible=False)
ax.set_title("What the forest relies on\n(permutation importance, city-grouped folds)", fontsize=12)
save(fig, "fig3_permutation_importance.png")

# ================================================================ 7. partial dependence + failure analysis
log("7. partial dependence and failure analysis")
fig, axes = plt.subplots(1, 3, figsize=(12, 4.2), sharey=True)
for ax, feat, lab in zip(axes, ["manufacturing_enterprises", "population_thousands", "temperature"],
                         ["Manufacturing enterprises (20+ workers)", "Population (thousands)", "Temperature (°F)"]):
    pdp = partial_dependence(m_all, d[RF_FEATURES], [feat], grid_resolution=60)
    ax.plot(pdp["grid_values"][0], pdp["average"][0], color=BLUE, lw=2)
    ax.plot(d[feat], np.full(len(d), ax.get_ylim()[0]), "|", color=MUTED, alpha=0.4, ms=8)
    ax.set_xlabel(lab); ax.set_title("")
axes[0].set_ylabel("Predicted SO2 (µg/m³)"); fig.suptitle("Partial dependence: flat beyond the data the forest has seen",
x=0.01, ha="left", fontweight="bold")
save(fig, "fig7_partial_dependence.png")

cm = L_tuned.groupby("city").agg(actual=("SO2", "mean"), predicted=("base", "mean"),
                                 manufacturing=("manufacturing_enterprises", "first"), population=("population_thousands",
"first"))
cm["error"] = cm.actual - cm.predicted
cm.sort_values("error").round(2).to_csv(os.path.join(OUT, "rf_city_errors_k0.csv"))
print("   Largest k=0 errors (actual - predicted, city mean):")
print(cm.reindex(cm.error.abs().sort_values(ascending=False).index).head(8).round(1).to_string())
fig, ax = plt.subplots(figsize=(7, 6))
lim = [0, max(cm.actual.max(), cm.predicted.max()) + 5]
ax.plot(lim, lim, color=MUTED, lw=1.2, ls="--"); ax.scatter(cm.predicted, cm.actual, s=45, color=BLUE, edgecolor=SURFACE,
linewidth=1.2, zorder=3)
for c, r in cm.iterrows():
    if abs(r.error) > 22: ax.annotate(c, (r.predicted, r.actual), xytext=(6, -13) if c == "Dallas" else (6, 4),
textcoords="offset points", fontsize=9, color=INK)
ax.set_xlabel("Forest prediction for the city (never saw it in training), µg/m³"); ax.set_ylabel("Actual city mean SO2,
µg/m³")
ax.set_title("Where the forest misses: a few heavy-SO2 cities"); ax.set_xlim(lim); ax.set_ylim(lim)
save(fig, "fig4_predicted_vs_actual.png")
es = cm.sort_values("error")
fig, ax = plt.subplots(figsize=(8.6, 9))
ax.barh(es.index, es.error, color=[BLUE if v >= 0 else ORANGE for v in es.error], height=0.65)
ax.axvline(0, color=INK2, lw=0.8); ax.set_xlabel("Actual minus predicted city mean SO2 (µg/m³)"); ax.grid(axis="y",
visible=False)
ax.set_title("City-level error, k = 0 (blue: forest too low, orange: too high)"); ax.tick_params(axis="y", labelsize=8)
save(fig, "fig5_city_errors.png")

# ================================================================ 8. one-line summary
r0 = tuned.loc[0]
print(f"\nSUMMARY  RF tuned (nested CV), new city k=0: pooled RMSE {r0.pooled_RMSE:.2f}, pooled R2 {r0.pooled_R2:.3f}"
      f"  (naive guess {naive_row['pooled_RMSE']:.2f})   k=1: {tuned.loc[1].pooled_RMSE:.2f}   k=2:
{tuned.loc[2].pooled_RMSE:.2f}")
log("done. Files in", OUT)
```

A5. so2_mixed_effects.py

```
"""
STTN327 SO2 project - Mixed-effects model (random intercept per city)
Reproduces the numbers in SO2_Group_Starter_Doc.docx.

SO2_ij = b0 + b1*temp + b2*wind + b3*precip + b4*precipdays + b5*manuf + b6*pop
         + g2*Q2 + g3*Q3 + g4*Q4 + u_i + e_ij,  u_i ~ N(0,tau2), e_ij ~ N(0,sigma2)
Fitted by REML (statsmodels MixedLM).  Evaluation: leave-one-city-out (LOCO).

Run:  python so2_mixed_effects.py [path/to/quarterly_SO2_data.csv]
Needs: pandas numpy statsmodels scipy scikit-learn
"""
import sys, warnings
import numpy as np, pandas as pd
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_factor
from scipy import stats
from sklearn.metrics import roc_auc_score
warnings.filterwarnings("ignore")

CSV = sys.argv[1] if len(sys.argv) > 1 else "quarterly_SO2_data/quarterly_SO2_data.csv"
d = pd.read_csv(CSV)
d["quarter"] = d["quarter"].astype(int)
d = d.sort_values(["city", "year", "quarter"]).reset_index(drop=True)
d["t"] = d.groupby("city").cumcount()          # 0..11 time order within city
d["Q2"] = (d.quarter == 2).astype(float); d["Q3"] = (d.quarter == 3).astype(float); d["Q4"] = (d.quarter == 4).astype(float)

XCOLS = ["temperature", "windspeed", "precipitation", "precipitationdays",
         "manufacturing_enterprises", "population_thousands", "Q2", "Q3", "Q4"]
FORMULA = "SO2 ~ " + " + ".join(XCOLS)
cities = list(d.city.unique())

def fit(data):
    m = smf.mixedlm(FORMULA, data, groups=data["city"]).fit(reml=True)
    return m

def index_of(m, data):
    """SO2 Index = fixed part only (no city effect)."""
    b = m.fe_params
    return b["Intercept"] + data[XCOLS].values @ b[XCOLS].values

# ---------------------------------------------------------------- 1. full fit
full = fit(d)
tau2, sigma2 = float(full.cov_re.iloc[0, 0]), float(full.scale)
print("=== Mixed-effects fit (all 41 cities) ===")
print(full.fe_params.round(3).to_string())
print(f"tau2 (between-city) = {tau2:.2f}   sigma2 (within-city) = {sigma2:.2f}   ICC = {tau2/(tau2+sigma2):.3f}")
fe_var = np.var(d[XCOLS].values @ full.fe_params[XCOLS].values, ddof=1)
print(f"Nakagawa R2: marginal = {fe_var/(fe_var+tau2+sigma2):.3f}  conditional = {(fe_var+tau2)/(fe_var+tau2+sigma2):.3f}")

# ---------------------------------------------------------------- 2. LOCO
rows = []      # one row per held-out quarter
for c in cities:
    tr, te = d[d.city != c], d[d.city == c].copy()
    m = fit(tr)
    t2, s2 = float(m.cov_re.iloc[0, 0]), float(m.scale)
    idx = index_of(m, te)
    te["index"] = idx
    te["naive"] = tr.SO2.mean()
    for k in (0, 1, 2, 3):
        if k == 0:
            pred = idx.copy()
        else:
            miss = (te.SO2.values[:k] - idx[:k]).mean()      # average miss on revealed quarters
            lam = t2 * k / (t2 * k + s2)                     # partial pooling (empirical Bayes)
            pred = idx + lam * miss
        te[f"pred{k}"] = pred
    rows.append(te)
L = pd.concat(rows)

def pooled(df, col, k):
    sub = df[df.t >= k]                                      # score only the remaining quarters
    err = sub.SO2 - sub[col]
    naive_err = sub.SO2 - sub["naive"]
    rmse = np.sqrt((err**2).mean())
    r2 = 1 - (err**2).sum() / (naive_err**2).sum()           # pooled R2 vs naive guess
    per_city = sub.assign(e2=err**2).groupby("city").e2.mean().pow(0.5).mean()
    return rmse, r2, per_city

print("\n=== Leave-one-city-out results ===")
nrm, _, ncity = pooled(L, "naive", 0)
print(f"Naive guess (mean of other 40 cities): pooled RMSE {nrm:.2f}, avg per-city RMSE {ncity:.2f}")
for k in (0, 1, 2, 3):
    rmse, r2, pc = pooled(L, f"pred{k}", k)
    print(f"k={k}: pooled RMSE {rmse:.2f}  pooled R2 {r2:.3f}  avg per-city RMSE {pc:.2f}")

# ---------------------------------------------------------------- 3. conformal band (new city, k=0)
L["miss"] = L.SO2 - L["index"]
g = L.groupby("city").miss
n, alpha = len(cities), 0.10
j = int(np.ceil((n + 1) * (1 - alpha)))                      # = 38
abs_scores = np.sort(g.apply(lambda x: x.abs().max()).values)
up = np.sort(g.max().values)[j - 1]                          # upper margin
lo = np.sort((-g.min()).values)[j - 1]                       # lower margin
print(f"\n=== Conformal band (n={n}, alpha={alpha}, j={j}) ===")
print(f"Symmetric band: Index +/- {abs_scores[j-1]:.2f}")
print(f"One-sided margins (38th smallest signed score): upper {up:.2f}, lower {lo:.2f}")
print('NOTE: the starter doc quotes 23.62 / 21.54 for the threshold rules; those two values are not reproduced by this
script.')
idx_full = pd.Series(index_of(full, d), index=d.index)
city_idx = idx_full.groupby(d.city).agg(["min", "max"])
print("Cities with Index <= T-upper (T=40) i.e. <=", round(40-up, 2), ":", int((city_idx["max"] <= 40 - up).sum()))
print("Cities with Index >= T+lower (T=20) i.e. >=", round(20+lo, 2), ":", int((city_idx["min"] >= 20 + lo).sum()))
```

```
# high/low classifier: city above/below median city mean SO2
cm = d.groupby("city").SO2.mean(); med = cm.median()
ci = L.groupby("city")["index"].mean()
y = (cm.loc[ci.index] > med).astype(int)
print(f"Median city SO2 = {med:.1f};  AUC = {roc_auc_score(y, ci):.2f};  accuracy (Index > {med:.1f}) = {((ci >
med).astype(int) == y).mean():.2f}")

# ---------------------------------------------------------------- 4. diagnostics
print("\n=== Diagnostics ===")
res = full.resid
d["resid"] = res.values
lagged = d.groupby("city").resid.shift(1); ok = lagged.notna()
lag = np.corrcoef(d.resid[ok], lagged[ok])[0, 1]
dw = (d.groupby("city").resid.diff()**2).sum() / (d.resid**2).sum()
print(f"Lag-1 correlation within city: {lag:.2f};  Durbin-Watson: {dw:.2f}")
print(f"Residual skew {stats.skew(res):.2f}, Shapiro-Wilk p = {stats.shapiro(res)[1]:.3f}")
groups = [x.values for _, x in d.groupby("city").resid]
print(f"Residual SD per city: {min(np.std(x, ddof=1) for x in groups):.2f}-{max(np.std(x, ddof=1) for x in groups):.2f};
Levene p = {stats.levene(*groups)[1]:.4f}")
re = pd.Series({c: v.iloc[0] for c, v in full.random_effects.items()})
print(f"City intercepts: Shapiro p = {stats.shapiro(re)[1]:.3f}; largest: {re.sort_values().tail(3).round(1).to_dict()}")
X = d[XCOLS].assign(const=1.0)
print("VIF:", {c: round(float(variance_inflation_factor(X.values, i)), 1) for i, c in enumerate(X.columns) if c in
('manufacturing_enterprises','population_thousands')})
print("corr(manufacturing, population) =", round(d.manufacturing_enterprises.corr(d.population_thousands), 2))
miss_city = (L.groupby("city").miss.mean()).sort_values()
print("Biggest under-predictions (city SO2 above Index):", miss_city.tail(3).round(1).to_dict())
```

A6. so2_worked_example.py: the San Francisco example (Figure 13)

```
"""
STTN327 SO2 project - Worked example: predict one city that was left out of the fit.

Fits the mixed-effects model on the other 40 cities, then predicts the held-out city
  k = 0 : no SO2 readings from the city   -> Index, with a 90% conformal band
  k = 1 : first quarter revealed          -> Index + lambda * (average miss)
and draws the picture used in the report and the video.

Run:  python so2_worked_example.py [path/to/quarterly_SO2_data.csv] [city]
Needs: pandas numpy statsmodels matplotlib
Writes: worked_example_<city>.png
"""
import sys, warnings
import numpy as np, pandas as pd
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
warnings.filterwarnings("ignore")

CSV = sys.argv[1] if len(sys.argv) > 1 else "quarterly_SO2_data/quarterly_SO2_data.csv"
CITY = sys.argv[2] if len(sys.argv) > 2 else "San Francisco"

d = pd.read_csv(CSV)
d["quarter"] = d["quarter"].astype(int)
d = d.sort_values(["city", "year", "quarter"]).reset_index(drop=True)
d["t"] = d.groupby("city").cumcount()
for q in (2, 3, 4):
    d[f"Q{q}"] = (d.quarter == q).astype(float)
XCOLS = ["temperature", "windspeed", "precipitation", "precipitationdays",
         "manufacturing_enterprises", "population_thousands", "Q2", "Q3", "Q4"]
FORMULA = "SO2 ~ " + " + ".join(XCOLS)


def fit(data):
    return smf.mixedlm(FORMULA, data, groups=data["city"]).fit(reml=True)


def index_of(m, data):
    b = m.fe_params
    return b["Intercept"] + data[XCOLS].values @ b[XCOLS].values


# ---- 90% conformal band (same rule as so2_mixed_effects.py): worst miss per city, 38th smallest of 41
worst, errs = {}, {}
for c in d.city.unique():
    tr, te = d[d.city != c], d[d.city == c]
    m = fit(tr)
    idx = index_of(m, te)
    worst[c] = np.abs(te.SO2.values - idx).max()
    errs[c] = np.sqrt(np.mean((te.SO2.values - idx) ** 2))
BAND = np.sort(list(worst.values()))[int(np.ceil((len(worst) + 1) * 0.90)) - 1]
median_err = float(np.median(list(errs.values())))

# ---- the held-out city
tr, te = d[d.city != CITY], d[d.city == CITY].sort_values("t")
m = fit(tr)
tau2, sigma2 = float(m.cov_re.iloc[0, 0]), float(m.scale)
idx = index_of(m, te)
lam = tau2 / (tau2 + sigma2)                       # k = 1
pred1 = idx + lam * (te.SO2.values[0] - idx[0])
actual = te.SO2.values
rmse0 = np.sqrt(np.mean((actual - idx) ** 2))
rmse1 = np.sqrt(np.mean((actual[1:] - pred1[1:]) ** 2))
inside = np.mean(np.abs(actual - idx) <= BAND)
print(f"{CITY}: k=0 RMSE {rmse0:.1f} (median city {median_err:.1f}); "
      f"k=1 RMSE on remaining quarters {rmse1:.1f}; band +/-{BAND:.2f}; "
      f"{inside:.0%} of quarters inside the band; lambda = {lam:.3f}")

# ---- figure
BLUE, ORANGE, INK, GREY = "#2A78D6", "#EB6834", "#0B0B0B", "#6B6A66"
plt.rcParams.update({"font.size": 14, "axes.spines.top": False, "axes.spines.right": False})
x = np.arange(12)
fig, ax = plt.subplots(figsize=(9.4, 5.1))

ax.fill_between(x, np.maximum(idx - BAND, 0), idx + BAND, color=BLUE, alpha=0.10, linewidth=0)
ax.plot(x, idx, color=BLUE, lw=2.6, marker="o", ms=5, zorder=3)
ax.plot(x[1:], pred1[1:], color=ORANGE, lw=2.6, marker="o", ms=5, zorder=4)
ax.scatter(x, actual, s=70, color=INK, zorder=5)
# the quarter we are allowed to see
ax.scatter([0], [actual[0]], s=260, facecolor="none", edgecolor=ORANGE, linewidth=2.2, zorder=6)
ax.text(0, actual[0] - 4.0, "first\nquarter\nseen", color=ORANGE, fontsize=11, ha="center", va="top", linespacing=1.0)

# direct labels instead of a legend
ax.text(11.25, idx[-1] + 1.5, "No city data\n(k = 0)", color=BLUE, fontsize=13, fontweight="bold", va="center")
ax.text(11.25, pred1[-1] - 0.5, "After one quarter\n(k = 1)", color=ORANGE, fontsize=13, fontweight="bold", va="center")
ax.text(11.25, actual[-1] - 7.0, "● Actual SO2", color=INK, fontsize=13, fontweight="bold", va="center")
ax.text(5.5, idx[4] + BAND - 4.0, "90% band for a city with no data", color=BLUE, fontsize=12, ha="center", alpha=0.9)

labels = [f"Q{q}" for q in te.quarter]
ax.set_xticks(x)
ax.set_xticklabels(labels, fontsize=12)
for i, yr in enumerate(sorted(te.year.unique())):
    ax.text(i * 4 + 1.5, -0.12, str(int(yr)), transform=ax.get_xaxis_transform(), ha="center", va="top",
            fontsize=13, color=GREY, fontweight="bold")
for b in (3.5, 7.5):
    ax.axvline(b, color="#DDDDD8", lw=1, zorder=0)
ax.set_xlim(-0.6, 13.4)
ax.set_ylim(0, None)
ax.set_ylabel("SO2 (µg/m³)")
ax.set_title(f"{CITY}, left out of the fit\n", loc="left", fontsize=16, fontweight="bold", color=INK, pad=22)
ax.text(0, 1.045, f"Typical error: {rmse0:.1f} µg/m³ with no city data  →  {rmse1:.1f} after one quarter",
        transform=ax.transAxes, fontsize=13, color=GREY, va="bottom")
```

```
fig.tight_layout()
out = f"worked_example_{CITY.replace(' ', '_')}.png"
fig.savefig(out, dpi=200)
print("saved", out)
```